Deep Learning for Computer Vision

L1 + L2 · Architectures and applications

4F12 Computer Vision · Michaelmas 2026 · Ayush Tewari



So far: problems with a clean model

Handout	Model you can write down
2 · Image structure	Harris: corners where $\lambda_1\lambda_2 - \kappa(\lambda_1 + \lambda_2)^2$ is large
3 · Projection	$\tilde{\mathbf{w}} = K\,[R \mid T]\,\tilde{\mathbf{X}}$
4 · Stereo	$\mathbf{p}'^{\top} E\,\mathbf{p} = 0, \quad \tilde{\mathbf{w}}'^{\top} F\,\tilde{\mathbf{w}} = 0$
4 · Structure	$E = U\Lambda V^{\top} \Rightarrow R,\ T$; triangulate by least squares

These methods use explicit models of image structure and geometry.

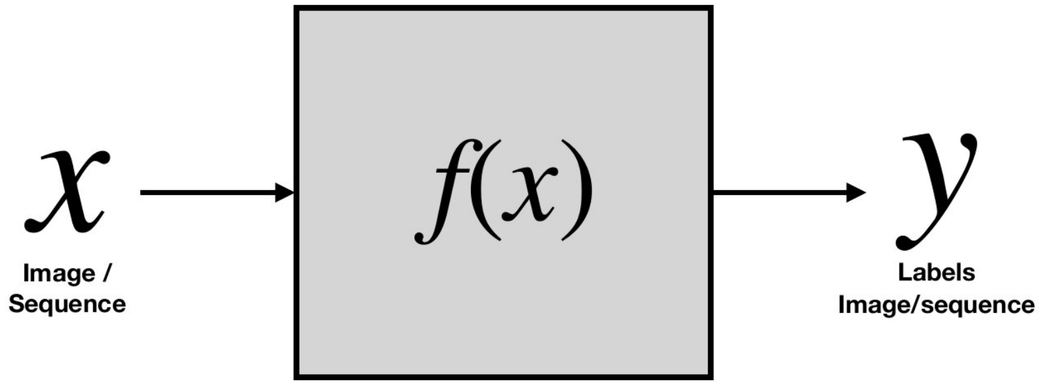
Many vision problems have no clean model

- Is there a bicycle in this image?
- Which pixels are the road?
- How far away is this wall, from one photo?
- Is this the same person after they walk behind a bus?

Questions for the deep learning lectures

	Question	Classical 4F12	Deep learning
L1	How do we compute features?	Handout 2: Canny, Harris, DoG, SIFT	CNNs and transformers
L2	What is where in the image?	No clean model; closest: deformable part models (HOG parts on springs, learned from labels)	Detection and segmentation
L3	Where is it in 3D?	Handouts 3–4: projection, stereo, structure from motion	Point tracking, learned matching, depth, DUSt3R, VGGT
L4	What can we learn without labels, and with language?	Bag of words, k-means	Self-supervised learning (MAE, DINO), CLIP, vision-language models

Supervised learning



What would make this problem supervised?

With labelled pairs $(x^{(i)}, y^{(i)})$, choose f from a hypothesis space $\mathcal{F}$:

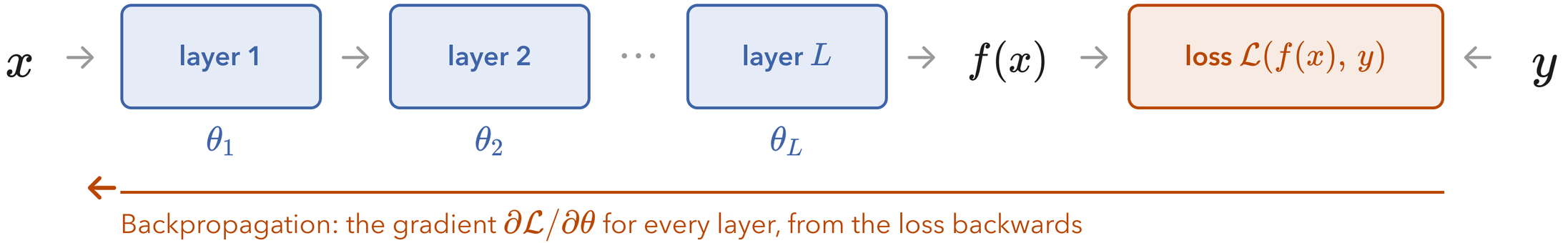
$$f^* = \arg\min_{f \in \mathcal{F}} \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}\big(f(x^{(i)}),\, y^{(i)}\big)$$

- Input: an $H \times W \times 3$ array of numbers
- Label: a one-hot vector $y \in \{0, 1\}^K$
- Loss: cross-entropy $-\sum_k y_k \log \hat{y}_k$
- Softmax: $\hat{y}_k = e^{z_k} / \sum_j e^{z_j}$, from the scores z

Figure: MLMI17 L0

Inside f: a deep network

f is a stack of layers, each with its own learned parameters.



Training: gradient descent changes $\theta_1, \ldots, \theta_L$ to reduce the average loss over the labelled pairs.

For each model we answer four questions:

1. **Data:** which labelled pairs (x, y) do we train on?
2. **Architecture:** which layers make up f?
3. **Loss:** how is $f(x)$ compared with y?
4. **Evaluation:** how do we measure success on new images?

First: architectures (CNNs, transformers). Then all four for two applications: detection (YOLO) and segmentation (SAM).

Assumed knowledge: backpropagation and stochastic gradient descent.

After MLMI17 L0.

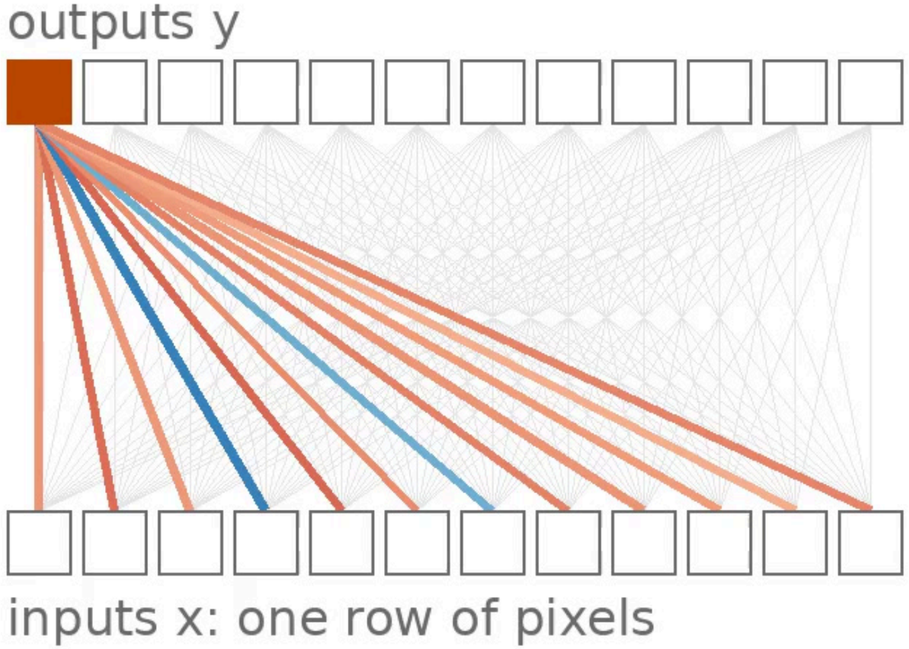
Architectures

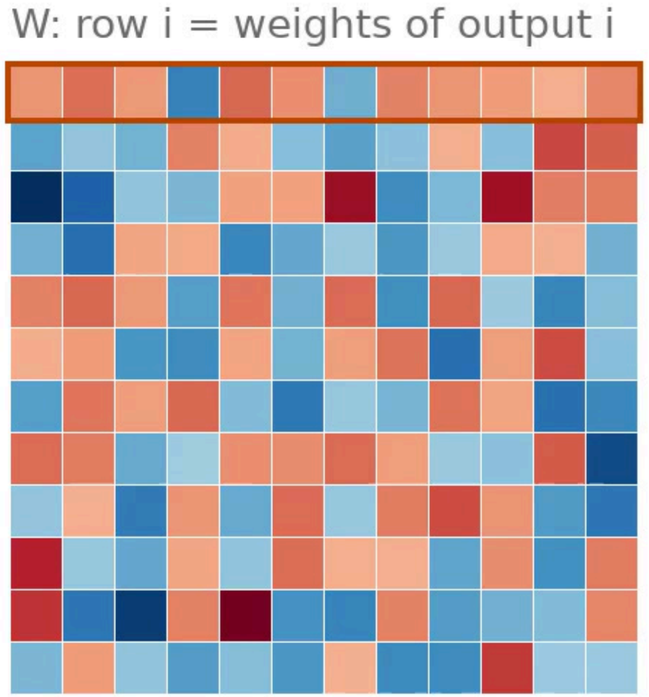
MLPs, CNNs and transformers

and the design choices behind systems in use today

A fully connected layer

$y_i = \sum_j W_{ij}\, x_j + b_i$: every output has its own weight on every input.





Weights

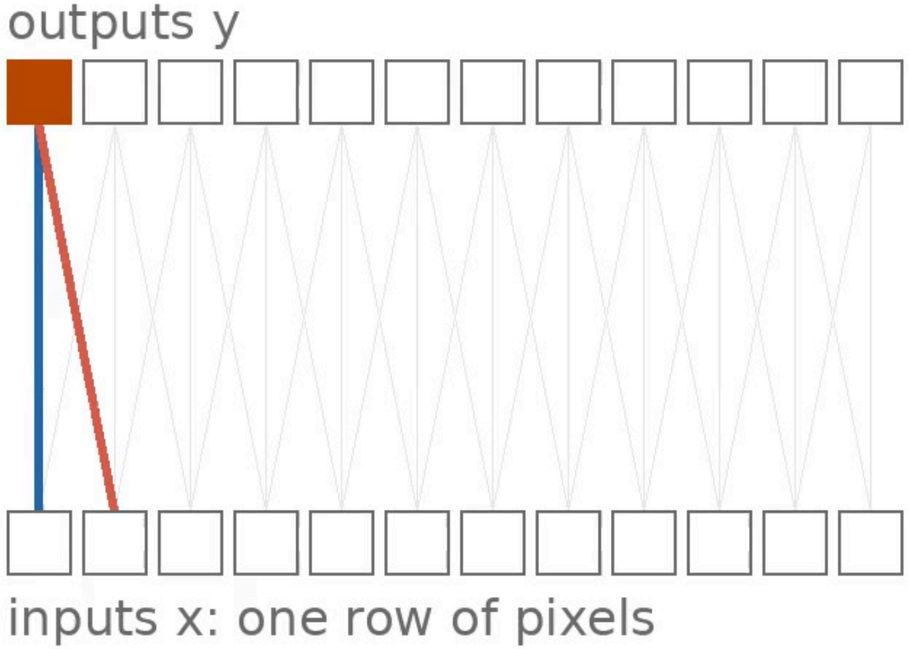
$N_{\text{in}} \times N_{\text{out}}$

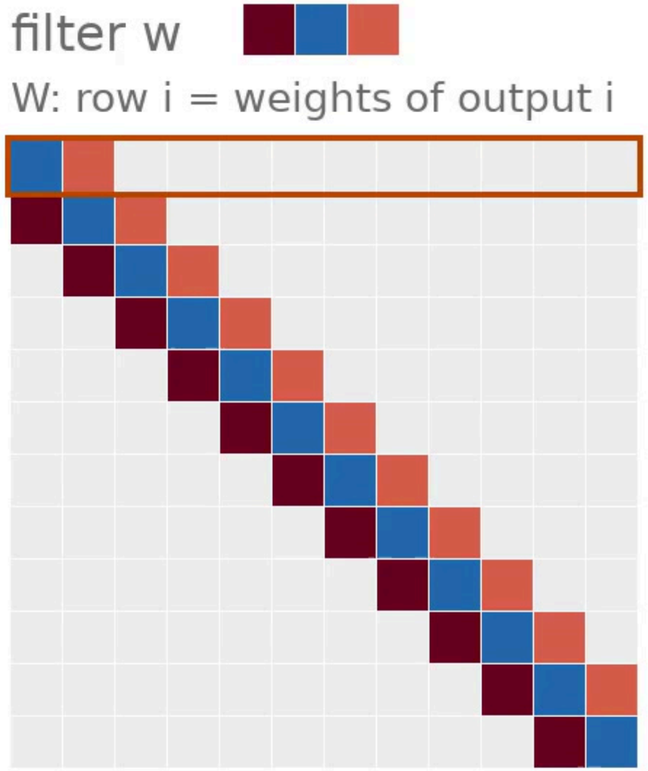
12 × 12 = 144 here

150,528 × 4,096 = 616,562,688 for a 224×224×3 image

A convolution layer

$y_i = \sum_{t=-1}^{1} w_t \, x_{i+t} + b$: every output applies the same small filter to its neighbours.





Weights

filter size only

3 here

64 filters of 3×3×3 = 1,728 for any image size

A fully connected layer with most weights set to zero (local) and the rest tied (shared).

Convolution: one output is one dot product

0	0	0	9	9	9
0	0	0	9	9	9
0	0	0	9	9	9
0	0	0	9	9	9
0	0	0	9	9	9
0	0	0	9	9	9

input (6×6): a vertical edge

$\otimes$

−1	0	1
−2	0	2
−1	0	1

kernel (3×3)

$=$

0	36	36	0
0	36	36	0
0	36	36	0
0	36	36	0

output (4×4)

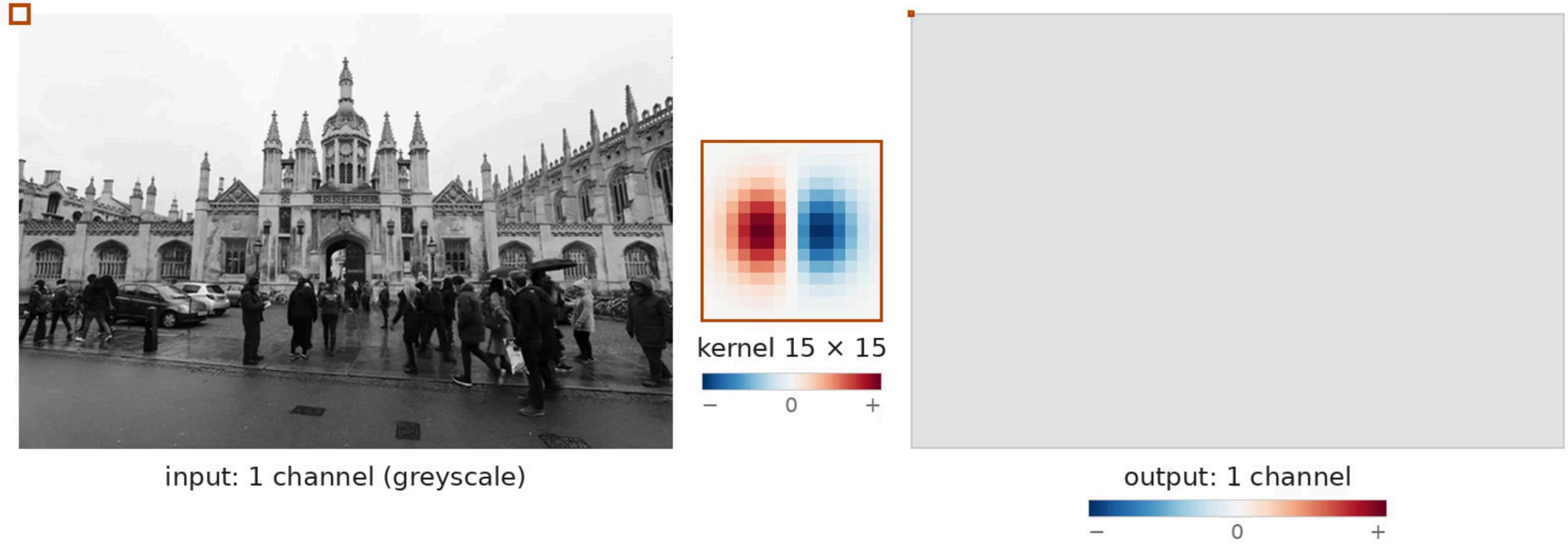
Window over columns 2–4:
$1 \cdot 9 + 2 \cdot 9 + 1 \cdot 9 = 36$

Window over columns 4–6 (all 9s): $-36 + 36 = 0$

Large response at the edge, zero on flat regions.

Deep-learning libraries compute cross-correlation (no kernel flip). The kernel is learned, so the flip doesn't matter.

Convolution on a real image



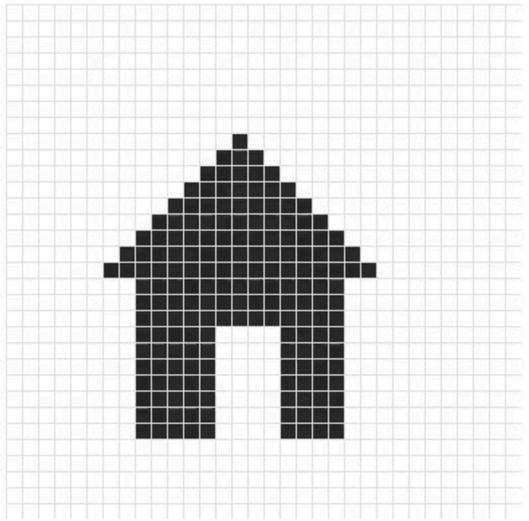
A 15×15 derivative-of-Gaussian kernel (Handout 2) slid over King's College. Red is positive, blue negative, white zero: a colour scale for one channel, not RGB.

A conv layer runs many kernels like this in parallel, and learns them.

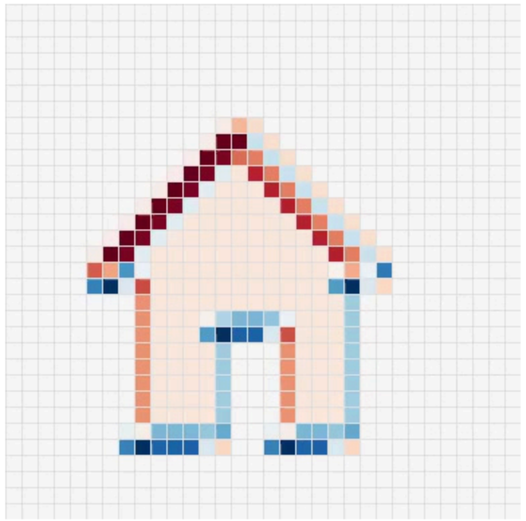
Equivariance: an inductive bias of convolution

A layer f is equivariant to a shift T if $f(Tx) = T\,f(x)$: shift the input and the output shifts with it.

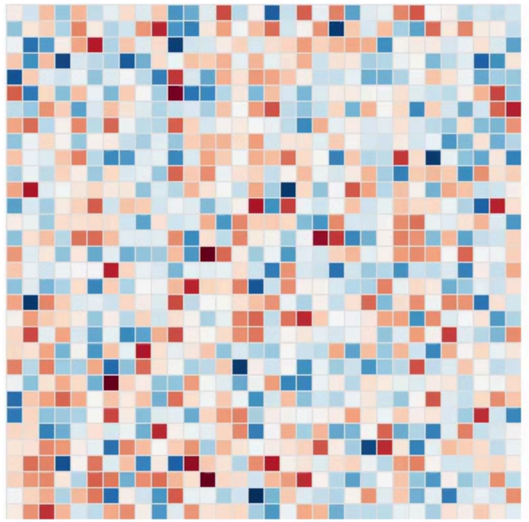
input Tx: shifted 0 px



conv layer: f(Tx)



MLP layer: f(Tx)



Conv layer: translation equivariance

One filter shared across positions.

MLP layer: no such structure

Learning a feature in one place does not automatically make it available elsewhere.

A CNN: three kinds of layers

A CNN repeats convolution, a nonlinearity and pooling, then classifies.



1. Convolution: many learned filters slide over the image; each gives one feature map. Next slide.

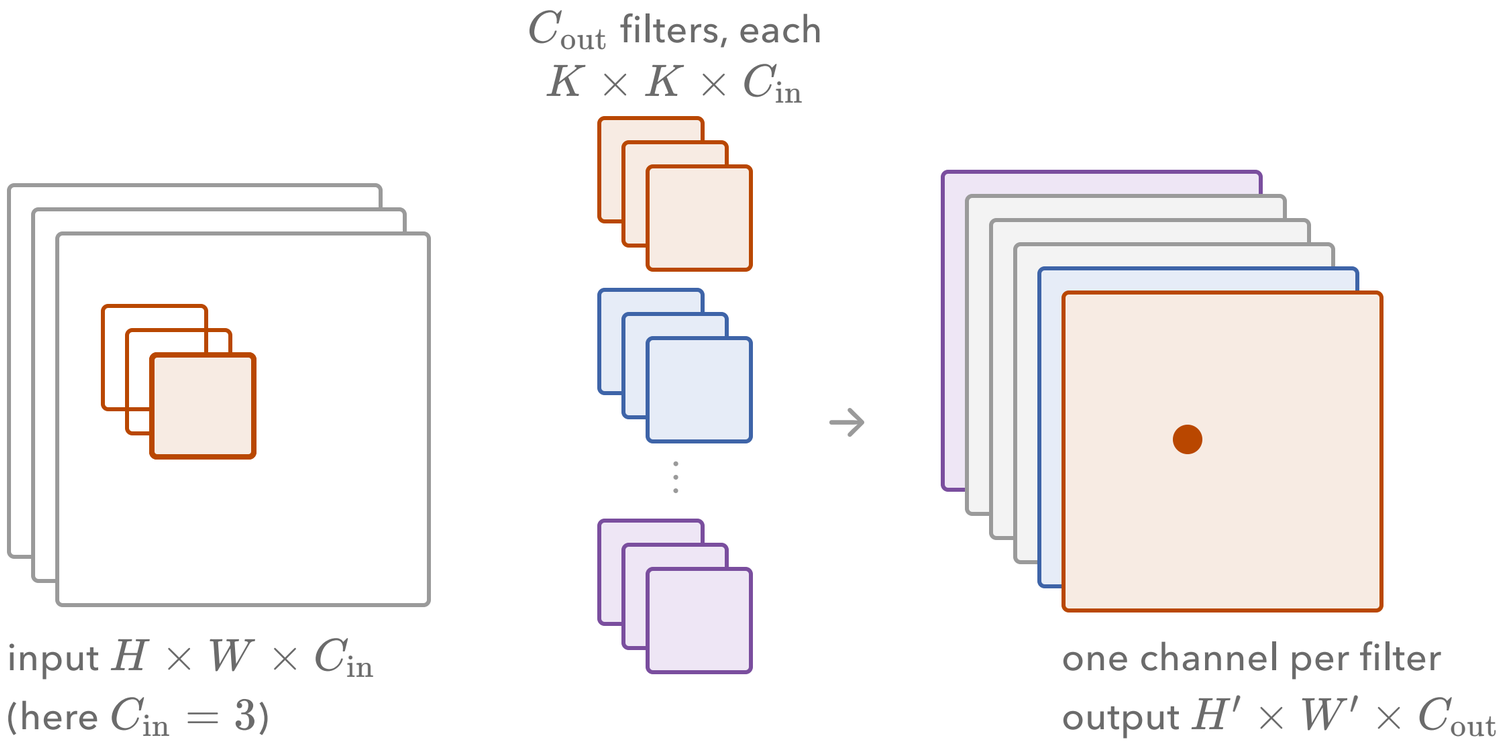
2. ReLU: a nonlinearity on every value; without it, stacked convolutions collapse into one convolution.

3. Pooling: shrinks the feature maps, so later layers see more of the image.

Then fully connected layers turn the last feature map into class scores. AlexNet (2012) stacks 5 convolution layers, with pooling after three of them, then 3 fully connected layers.

Krizhevsky, Sutskever & Hinton, NeurIPS 2012.

From one filter to a layer



1. One filter covers $K \times K$ pixels and **all** C_{in} input channels. At one position it gives one number.

2. Slide it over the image: one output channel.

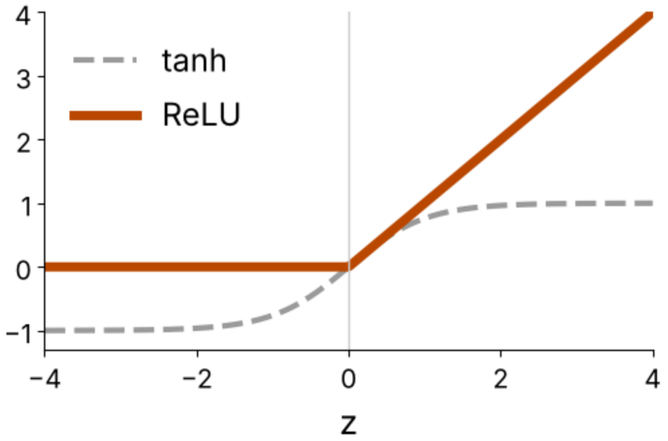
3. A layer has C_{out} filters, each with its own weights: C_{out} output channels.

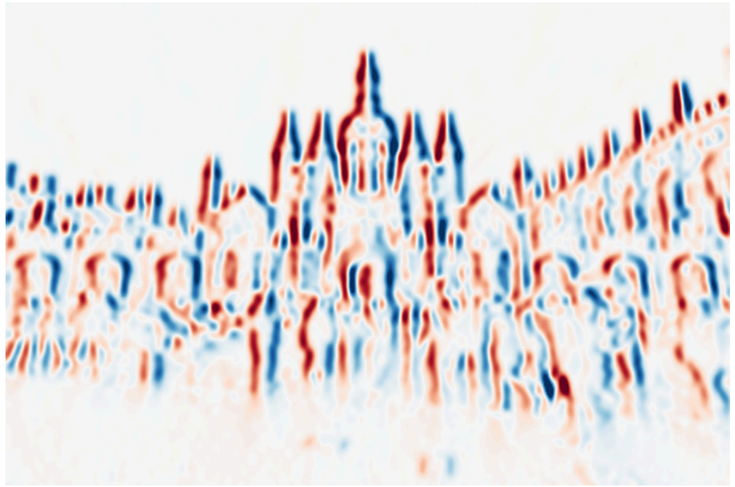
One filter: $K^2 C_{\text{in}}$ weights + 1 bias
Layer: $C_{\text{out}}(K^2 C_{\text{in}} + 1)$ parameters

e.g. $K = 3, C_{\text{in}} = 3, C_{\text{out}} = 64$:
1,728 weights + 64 biases

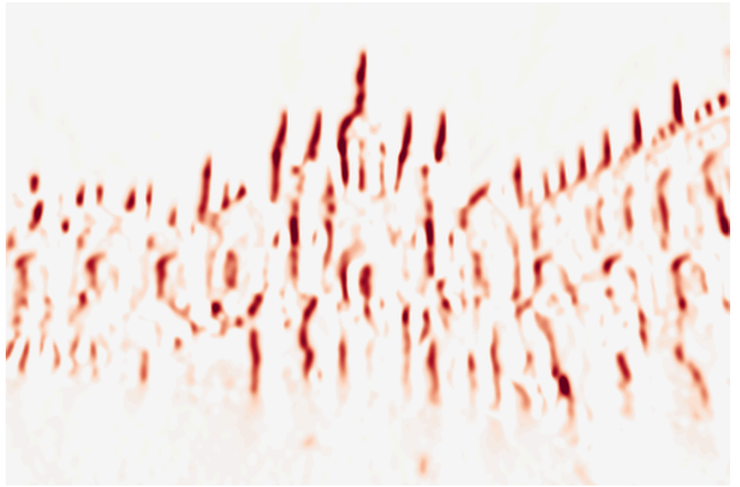
ReLU: a nonlinearity after every convolution

$\mathrm{ReLU}(z) = \max(0, z)$, applied to every value of the feature map





conv output: red positive, blue negative

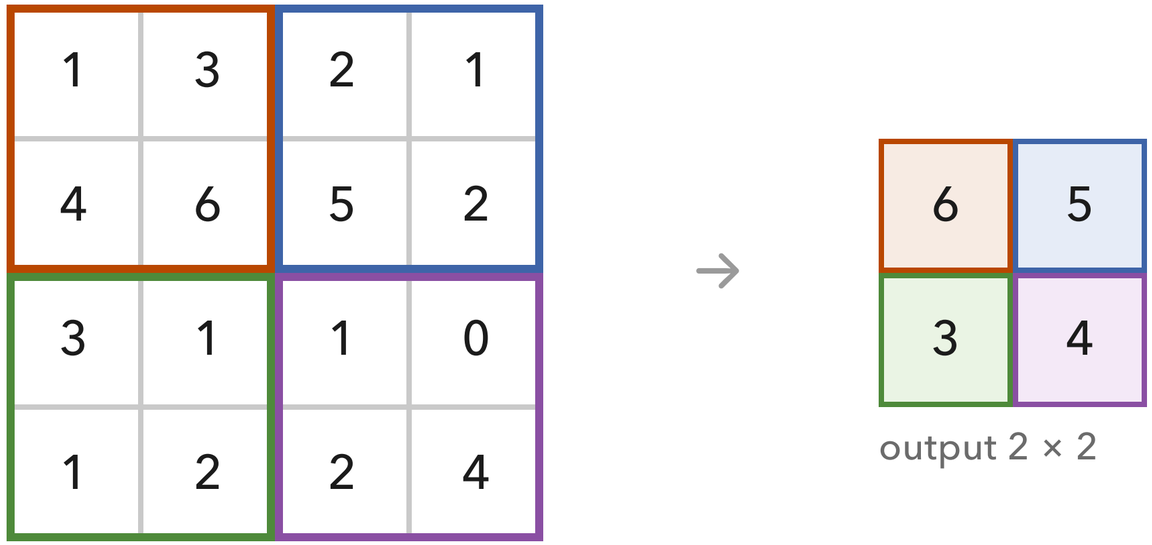


after ReLU: negatives set to zero

- Without it, stacked convolutions collapse into a single convolution
- It acts on each value separately, so the layer stays equivariant

Nair & Hinton, ICML 2010; Krizhevsky, Sutskever & Hinton, NeurIPS 2012

Pooling, step by step



input 4 × 4

output 2 × 2

2 × 2 max-pool, stride 2: keep the largest value in each window

1. Output size from the same rule: $(4-2)/2+1=2$

2. No weights to learn; works on each channel separately

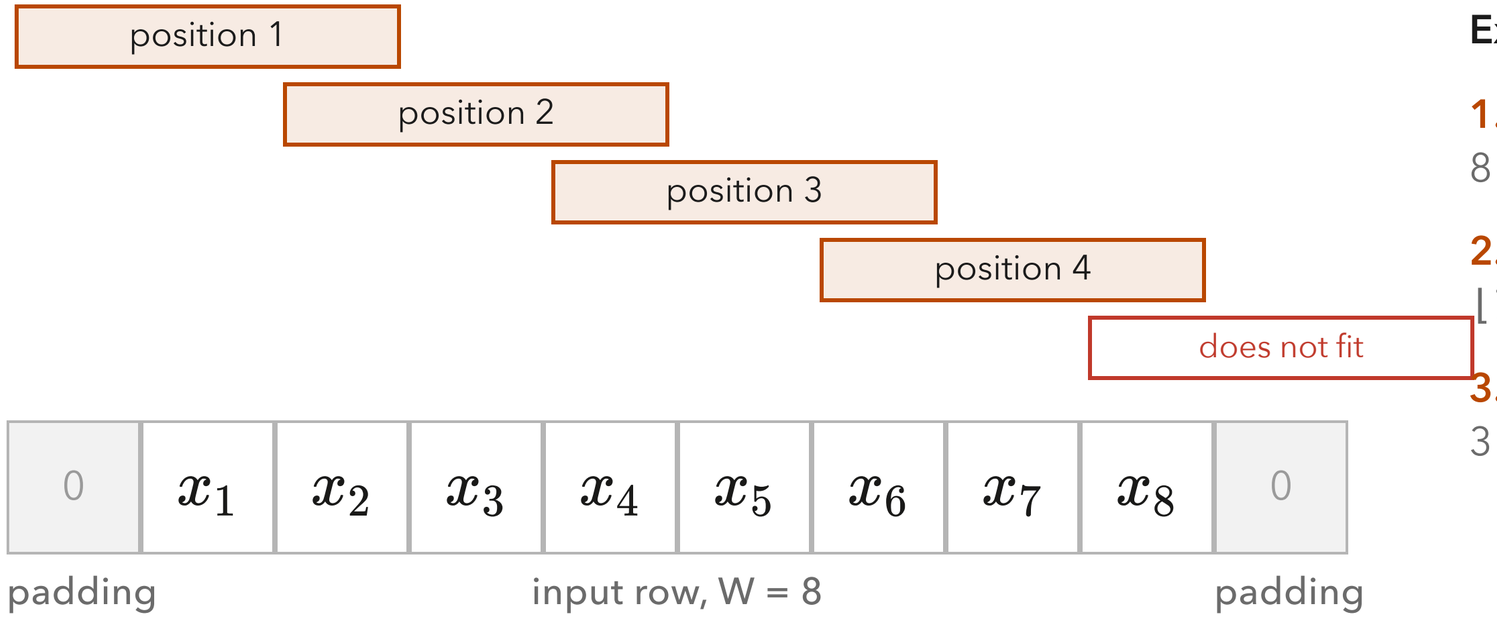
3. Small shifts of the input often leave the maxima unchanged

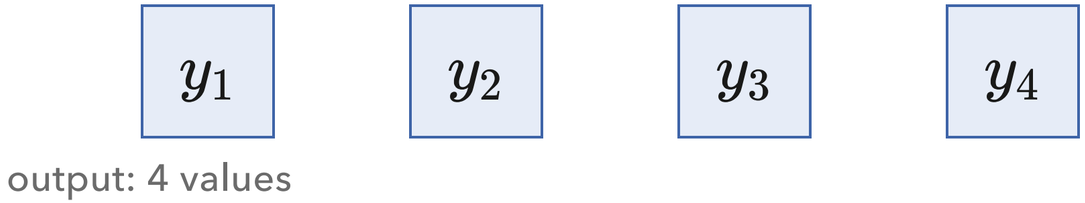
4. Halving the resolution doubles the jump, so later layers see much more of the image

Strided convolutions do the same job with learned weights (ResNet; the ViT patch embedding).

Output size, step by step

A filter of width K slides along W pixels padded with P zeros at each end, S pixels at a time.





Example: W = 8, K = 3, P = 1, S = 2

1. Room to slide: W + 2P − K = 7
8 + 2 − 3

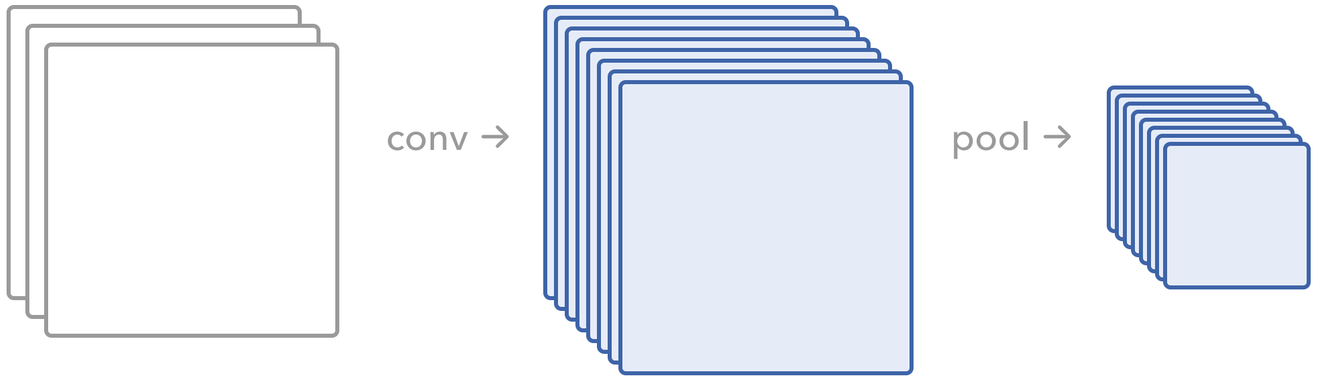
2. Whole steps of S = 2 in it: 3
⌊7 / 2⌋; the half step is dropped

3. Plus the first position: 4 outputs
3 + 1

$$W' = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

Conv layer arithmetic: a worked example

Input	32 × 32 × 3
Conv layer	16 filters, 5 × 5, stride 1, padding 2
Then	2 × 2 max-pool, stride 2



1. Output width and height
$(32 - 5 + 2 \cdot 2)/1 + 1 = 32$

2. Output depth = number of filters
32 × 32 × 16

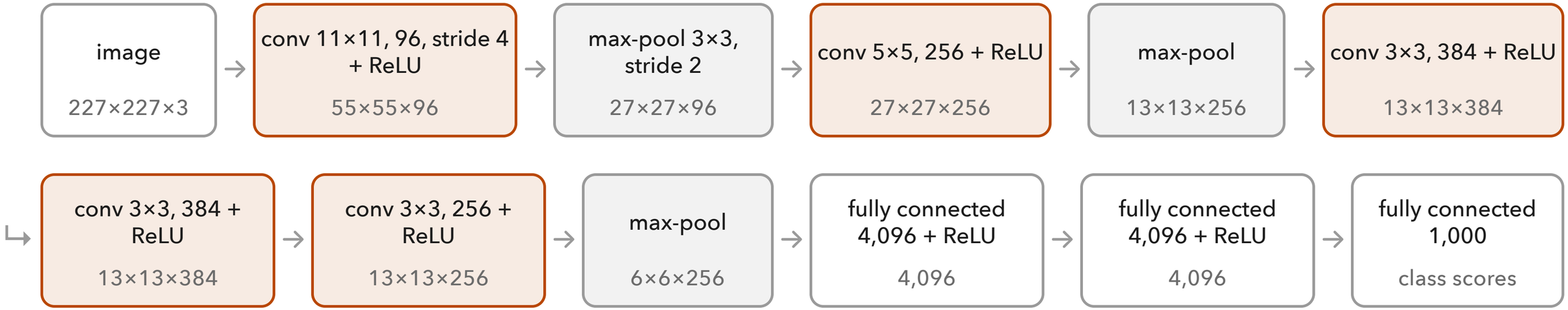
3. Weights in one filter, plus its bias
$5 \cdot 5 \cdot 3 + 1 = 76$

4. Parameters in the layer
$76 \times 16 = 1{,}216$

5. 2×2 max-pool, stride 2: no parameters
$(32 - 2)/2 + 1 = 16$, so 16 × 16 × 16

A CNN: convolution, ReLU, pooling, repeated

AlexNet (2012), the network that started the deep-learning era in vision:

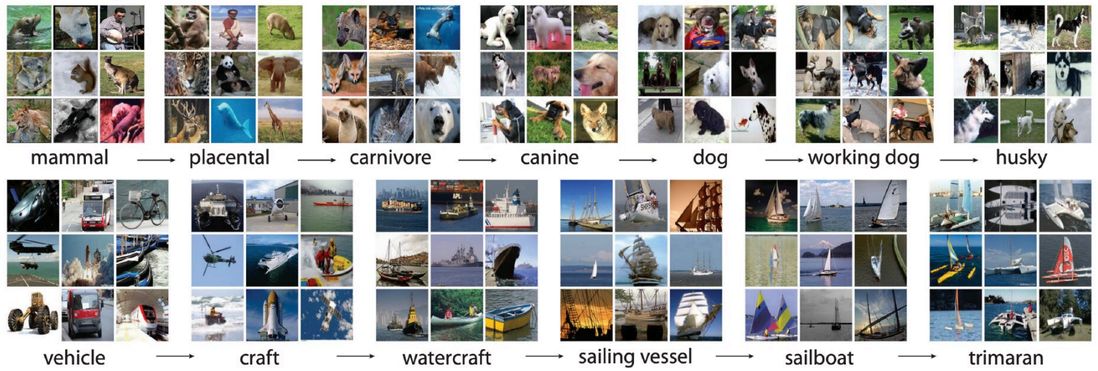


- Spatial size shrinks (55 → 27 → 13 → 6) while channels grow (96 → 256 → 384)
- The fully connected layers turn the final 6×6×256 map into 1,000 class scores

convolution + ReLU pooling fully connected

Krizhevsky, Sutskever & Hinton, NeurIPS 2012

ImageNet and AlexNet



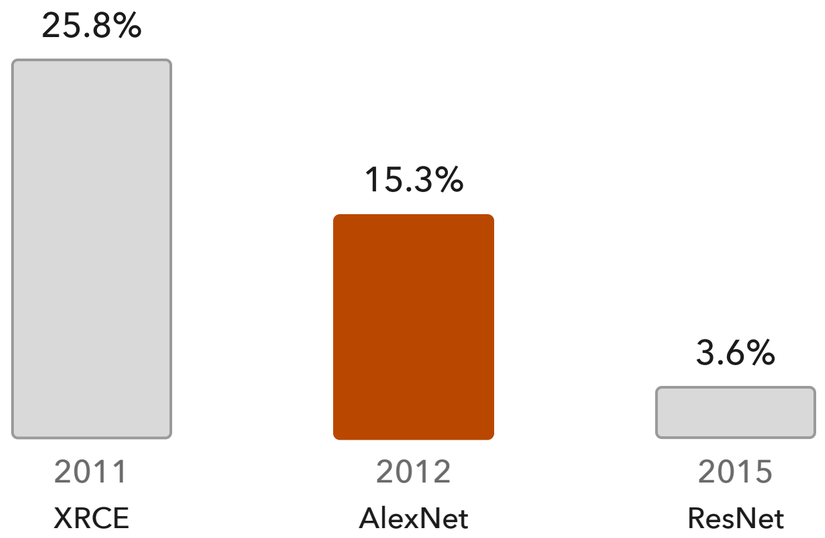
ImageNet [Deng et al., CVPR 2009]
>14M images with human annotations!

ImageNet: 1.2M training images in 1,000 classes.

AlexNet won 2012 with 15.3% top-5 error; the runner-up had 26.2%.

Trained on two GPUs with ReLU, dropout and data augmentation (crops, flips, colour).

ImageNet classification, top-5 error of the winning entry



Data augmentation: new training images, same label

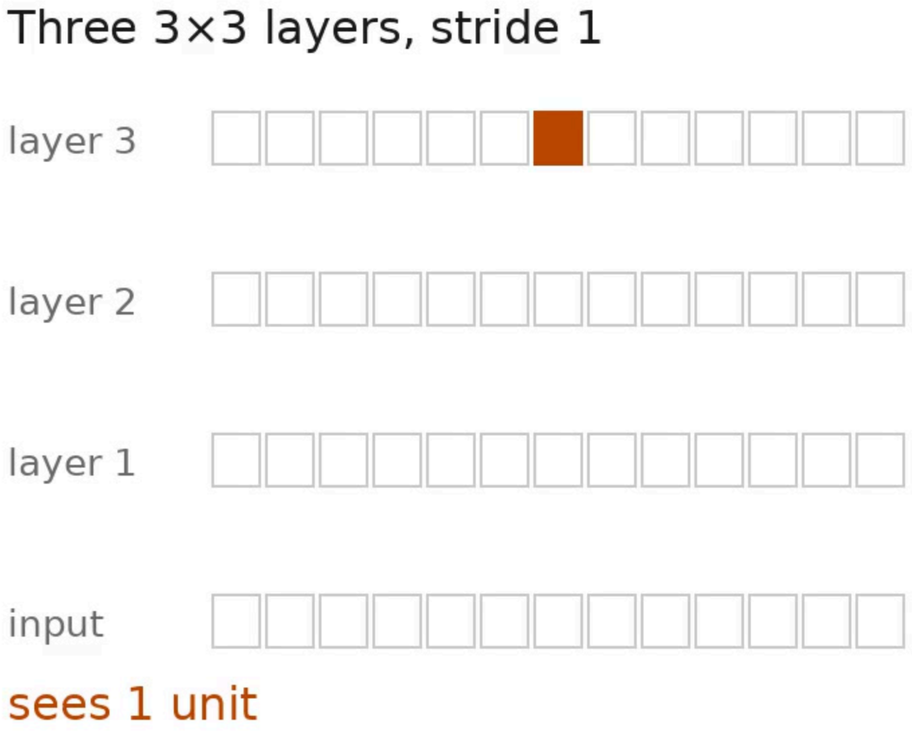


original, crop box | random 224×224 crop | horizontal flip | colour change

Deng et al., CVPR 2009; Russakovsky et al., IJCV 2015; Krizhevsky et al., NeurIPS 2012; He et al., CVPR 2016; figure: MLMI17 L0

Receptive field grows with depth

Receptive field: the input pixels that can affect one unit. One row of pixels shown.



Each 3×3 layer adds a pixel on each side: 1 → 3 → 5 → 7

After a stride-2 layer, neighbours are 2 pixels apart, so the next 3×3 layer adds 2 × 2 = 4

In general: $r_l = r_{l-1} + (k_l - 1)\, j_{l-1}$, where the jump j is the product of the strides so far

Deeper networks are hard to train

AlexNet has 8 layers.

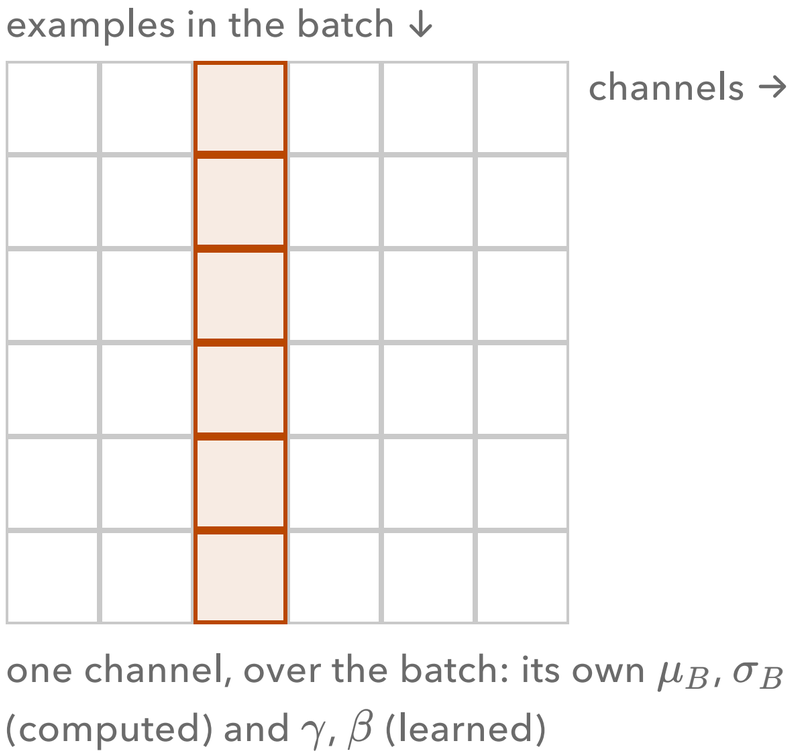
With deeper networks, learning gets harder.

Fix 1: batch normalisation

1. For each channel, take the mean μ_B and standard deviation σ_B of its activations x over the mini-batch (and all positions)

2. Normalise: $\hat{x} = (x - \mu_B)/\sigma_B$, so the channel has mean 0 and variance 1

3. Rescale and shift: $y = \gamma\hat{x} + \beta$



one channel, over the batch: its own μ_B, σ_B (computed) and γ, β (learned)

Transformers use LayerNorm instead: the same steps over the channels of one token (later).

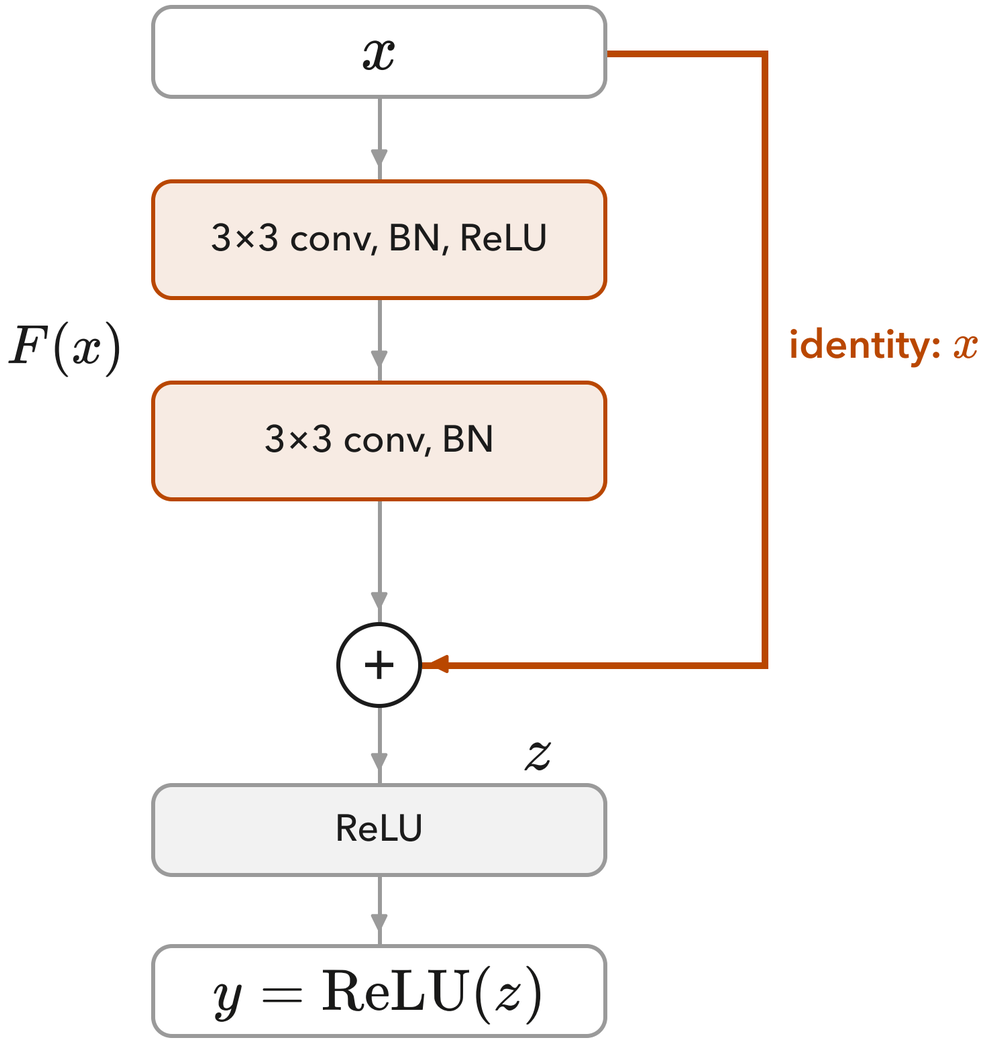
Simonyan & Zisserman, ICLR 2015; Ioffe & Szegedy, ICML 2015

Fix 2: residual connections

$z = F(x) + x$: the block learns a correction F to its input. When $F(x) = 0$, the sum gives $z = x$.

The gradient through the sum has a direct path:
$\partial z/\partial x = \partial F/\partial x + I$

ResNets won ImageNet 2015 with up to 152 layers.



He et al., CVPR 2016

Beyond local filters

Convolution

Same weights at every position. Each output sees a fixed small window.

Attention

Weights depend on the content. Each patch chooses what to look at, anywhere in the image.



Attention maps of a trained ViT for four query patches (coloured dots): each attends to the rest of its object.

DINO: Caron et al., ICCV 2021 (figure: MLMI17 L1)

ViT: cut the image into patches



224×224 image, 16×16 patches → 14×14 = 196 patches.

Each patch is treated like a word in a sentence.

ViT: each patch becomes a token



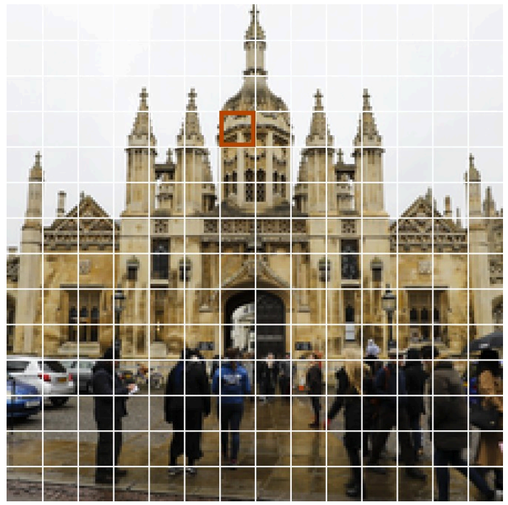


flatten: 16·16·3 = 768 numbers

token
$z \in \mathbb{R}^D$

Multiply by a learned matrix: $z = E\,x + b$, where $E \in \mathbb{R}^{D \times 768}$.
D is the chosen token width. ViT-B uses $D = 768$.

ViT: add a position code



original

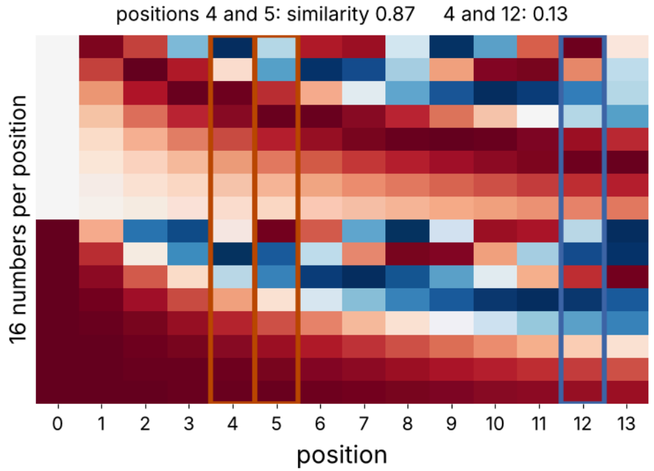


patches shuffled

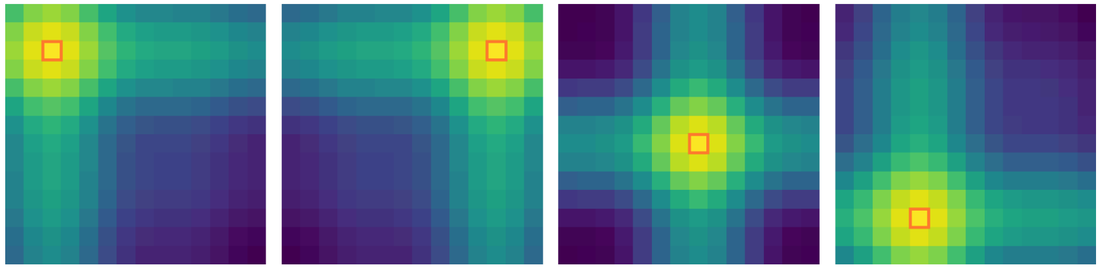
Nothing in a token says where its patch came from. So to a ViT these two images are the same set of tokens.

Fix: add a position code to each token before the first block:

$$z_i = E\,x_i + b + p_i$$



Each position gets its own pattern of numbers p_i, as long as a token. Nearby positions get similar patterns.



In 2D: one patch's code (orange) against all others. Fixed sine-wave codes shown; ViT learns similar ones.

Vaswani et al., NeurIPS 2017; Dosovitskiy et al., ICLR 2021

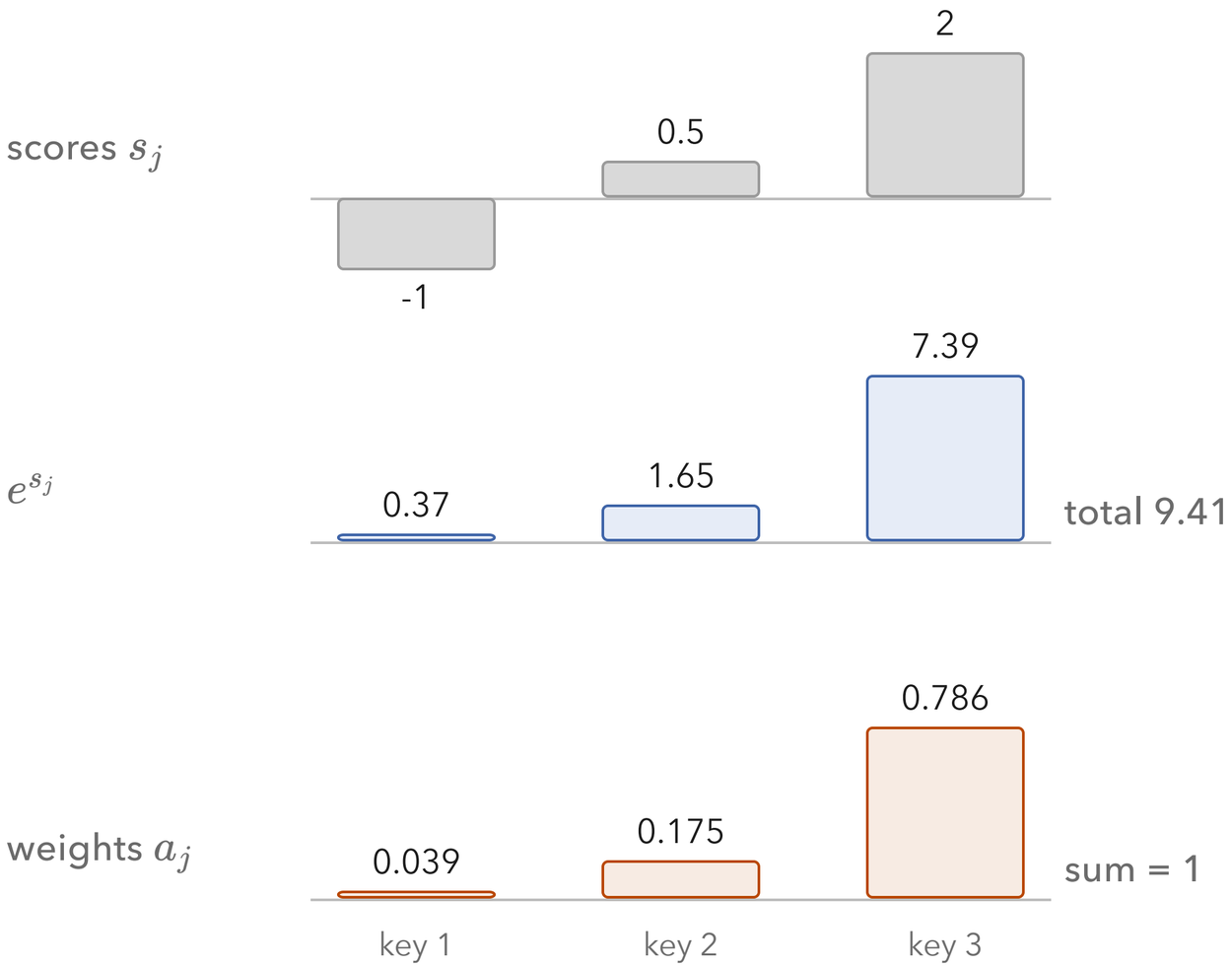
Attention: a soft lookup

Each token asks a question of every other token, and takes a weighted average of their answers.



Query: what this token is looking for. **Key**: what each token offers to be matched. **Value**: what it passes on.

From match scores to weights



1. Score each key with a dot product, $s_j = q \cdot k_j$: large when the key points the same way as the query, negative when it points away

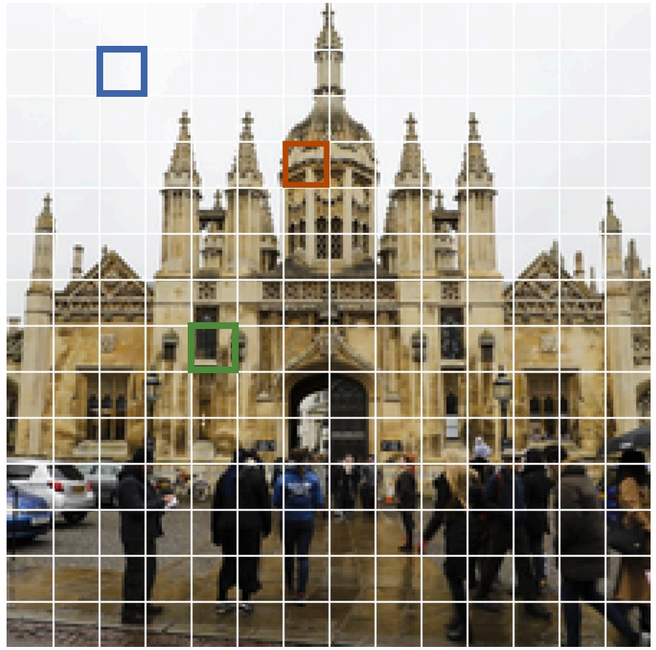
2. Softmax turns the scores into weights that are positive and add up to 1:

$$a_j = \frac{e^{s_j}}{\sum_m e^{s_m}}$$

The output is the weighted average of the values, $y = \sum_j a_j v_j$.

Scores are first divided by $\sqrt{D}$, where D is how many numbers each query and key has. This keeps them in a sensible range when the vectors are long.

A worked example: three tokens



three of the 196 patch tokens

	key k	value v = (sky, building)
sky	$(0,\ 1)$	$(1,\ 0)$
tower	$(\ln 3,\ 0)$	$(0,\ 1)$
wall	$(\ln 2,\ 1)$	$(0,\ 1)$

The tower's query: $q = (\sqrt{2},\ 0)$, with $D = 2$ numbers per vector

What does the tower token output?

The tower's output, step by step



sky

$k = (0,\ 1)$
$v = (1,\ 0)$



tower

$k = (\ln 3,\ 0)$
$v = (0,\ 1)$



wall

$k = (\ln 2,\ 1)$
$v = (0,\ 1)$

query $q = (\sqrt{2},\ 0)$

	sky	tower	wall
score	0	1.10	0.69
weight	$\frac{1}{6}$	$\frac{1}{2}$	$\frac{1}{3}$

1. Score: the dot product of the query with each key

$q \cdot k_{\text{sky}} = \sqrt{2} \cdot 0 + 0 \cdot 1 = 0$
$q \cdot k_{\text{tower}} = \sqrt{2} \cdot \ln 3 + 0 \cdot 0 = \sqrt{2} \ln 3$
$q \cdot k_{\text{wall}} = \sqrt{2} \cdot \ln 2 + 0 \cdot 1 = \sqrt{2} \ln 2$

2. Divide by $\sqrt{D} = \sqrt{2}$ to get the scores

$s = (0,\ \ln 3,\ \ln 2) \approx (0,\ 1.10,\ 0.69)$

3. Softmax: $a_j = e^{s_j} / \sum_m e^{s_m}$

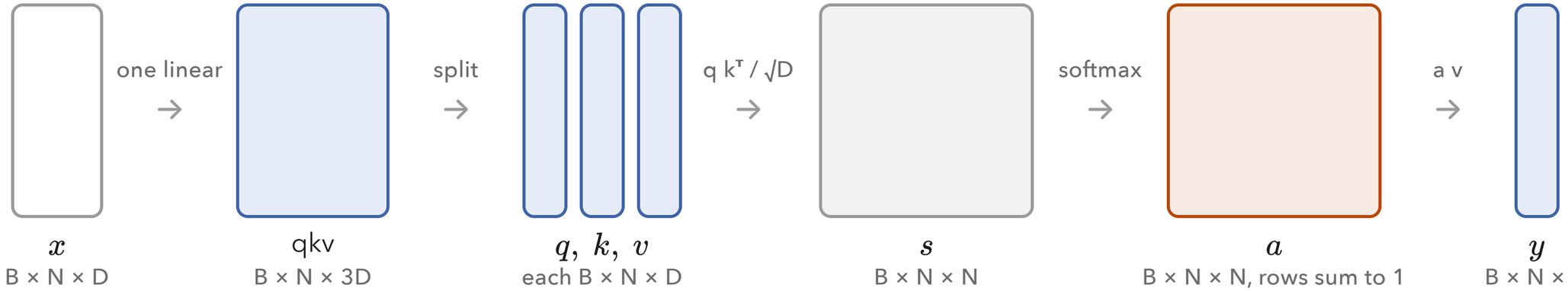
$$a = \frac{(e^0,\ e^{\ln 3},\ e^{\ln 2})}{1 + 3 + 2} = \frac{(1,\ 3,\ 2)}{6} = (\tfrac{1}{6},\ \tfrac{1}{2},\ \tfrac{1}{3})$$

4. Weighted average of the values

$y = \frac{1}{6}(1,0) + \frac{1}{2}(0,1) + \frac{1}{3}(0,1)$
$= (\frac{1}{6},\ \frac{1}{2} + \frac{1}{3}) = (\frac{1}{6},\ \frac{5}{6})$

Five-sixths ‘building’: the tower attends most to itself, then to the wall of the same building, least to the sky.

Self-attention as a library computes it




```
qkv = self.qkv(x)              # nn.Linear(D, 3D)
q, k, v = qkv.chunk(3, dim=-1)
s = q @ k.transpose(-2, -1) * D ** -0.5
a = s.softmax(dim=-1)
y = a @ v
```

x holds a batch: B images, N tokens each, D numbers per token.

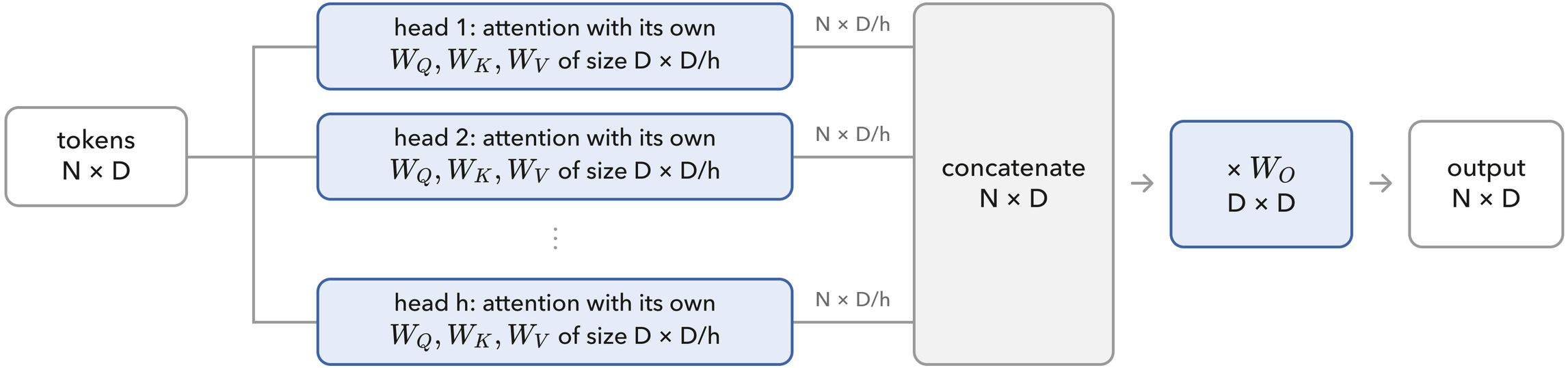
One linear layer makes q, k and v together; then it is split three ways.

@ is a batched matrix multiply: one $N \times N$ score matrix per image.

The last three lines can be replaced by `F.scaled_dot_product_attention(q, k, v)`.
PyTorch can use FlashAttention to avoid storing the full $N \times N$ matrix.

Multi-head attention

Run h attention lookups in parallel on narrower vectors, then combine them.



What changes from one head

1. Each head makes its own q, k, v with D/h numbers instead of D, so its scores are divided by $\sqrt{D/h}$.

2. Each head has its own $N \times N$ attention weights, so heads can attend to different things: nearby texture, the rest of the object, a matching part.

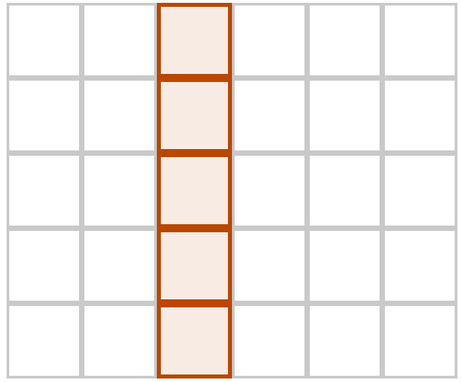
3. The h outputs are concatenated back to D numbers per token and mixed by one more matrix, W_O.

ViT-B: $D = 768$, $h = 12$ heads of 64 dimensions.

Vaswani et al., NeurIPS 2017

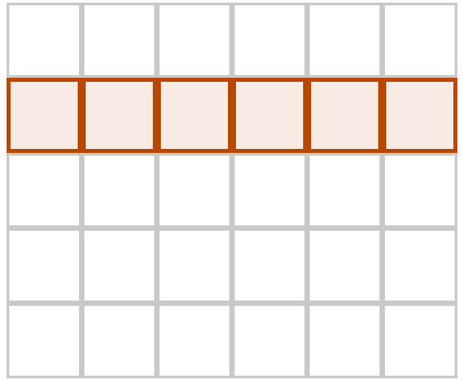
LayerNorm, step by step

examples or tokens ↓



channels →

BatchNorm: one channel, over the examples in a batch



LayerNorm: one token, over its channels

One token with 4 channels: $x = (2,\ 4,\ 6,\ 8)$

1. Mean: $\mu = 5$
2. Variance: $\sigma^2 = \frac{1}{4}(9 + 1 + 1 + 9) = 5$
3. Normalise: $\hat{x} = (x - \mu)/\sqrt{\sigma^2 + \epsilon} \approx (-1.34,\ -0.45,\ 0.45,\ 1.34)$
4. Rescale with learned γ, β, one pair per channel: $y = \gamma \odot \hat{x} + \beta$

ViT: the transformer block

1. Attention mixes information across tokens

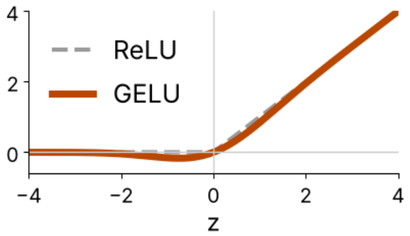
Each token gathers a weighted combination of information from other patches.

2. The MLP mixes channels within each token

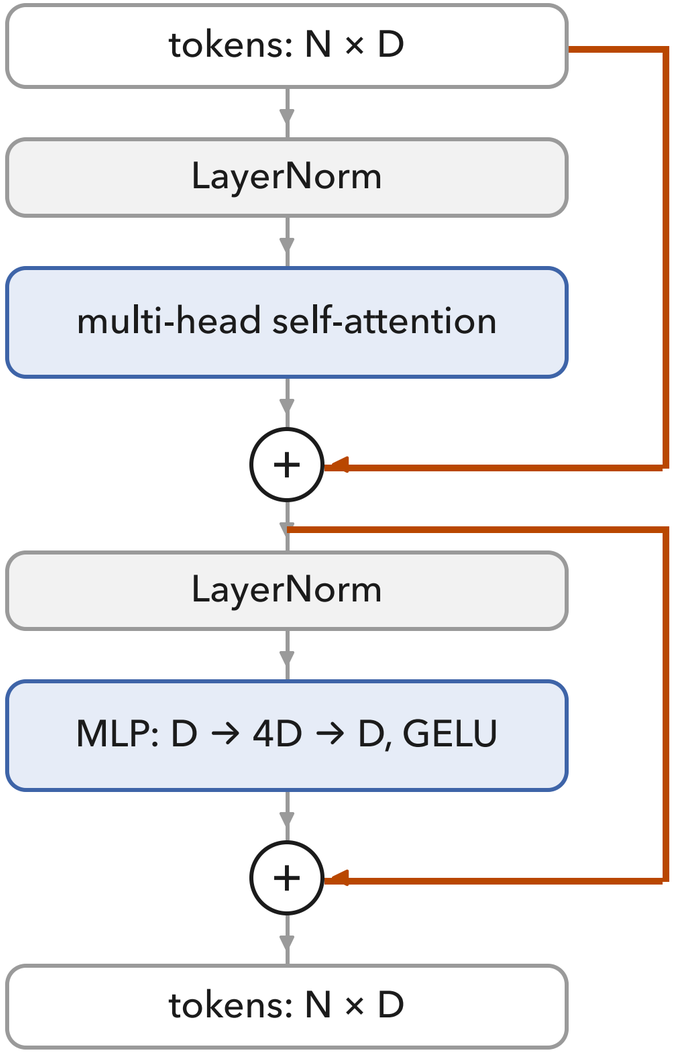
It applies the same nonlinear network to every token.

LayerNorm precedes each part.
Residual connections add each update to the input.

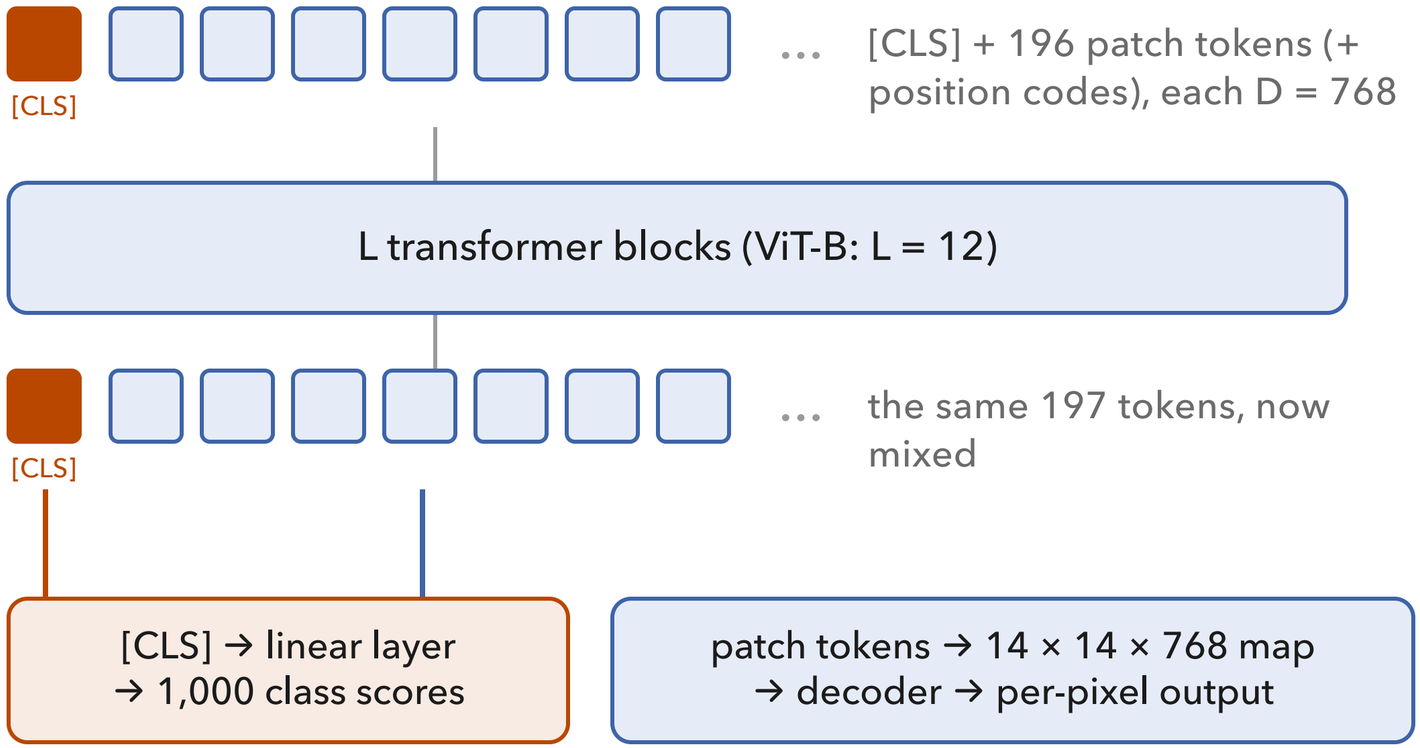
The token array stays $N \times D$.



GELU: a smooth version of ReLU, $\mathrm{GELU}(z) = z\,\Phi(z)$, where Φ is the standard normal CDF. Small negative values are damped, not cut to zero.



ViT: stack the blocks and read out



[CLS]: one extra learned token added at the start. Through attention it gathers information from every patch.

1. Classification: read the [CLS] output, one linear layer to class scores

2. Dense tasks (L2, L3): keep all patch outputs, reshape them back into a 14 × 14 map and decode

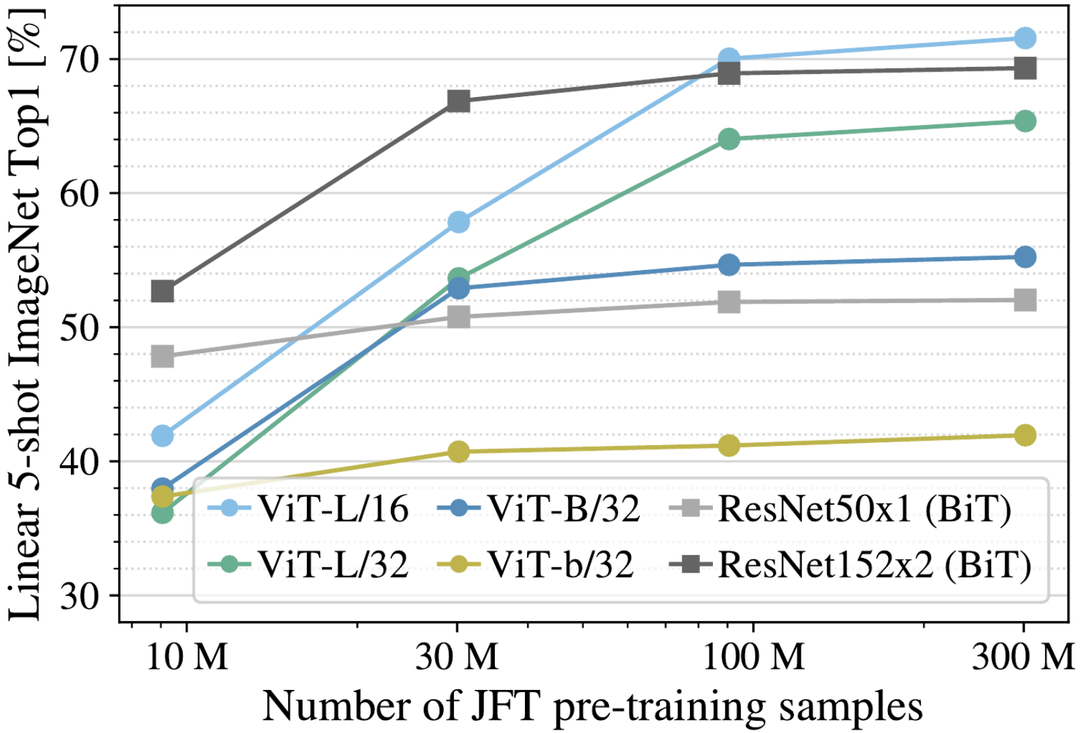
Many newer models average the patch outputs instead of using [CLS].

Dosovitskiy et al., ICLR 2021

CNN vs ViT

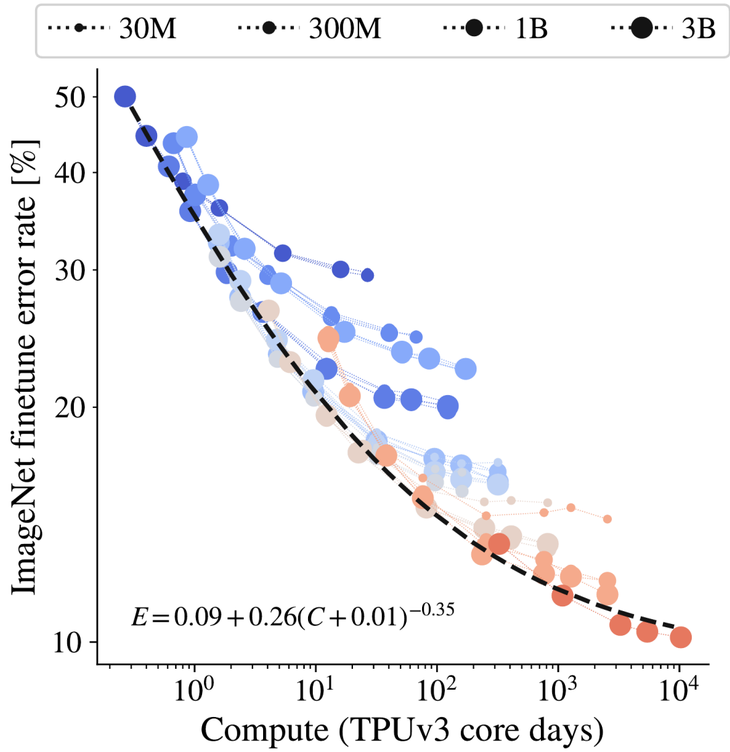
	CNN	ViT
Locality	built in	learned
Receptive field	grows with depth	global from layer 1
Position	implicit	positional encoding
Spatial mixing cost (fixed channel width)	$\propto HWK^2$	$\propto N^2$

Data and compute scaling



In the original ViT experiments, ResNets performed better with less pretraining data, while large ViTs performed better with more data.

Dosovitskiy et al., ICLR 2021, Fig. 4



Colours: different ViT models.
Marker size: pretraining images (B = billion).

ViT transfer performance improves when model size, training data and compute scale together.

Zhai et al., CVPR 2022, Fig. 2 (left panel)

Two applications

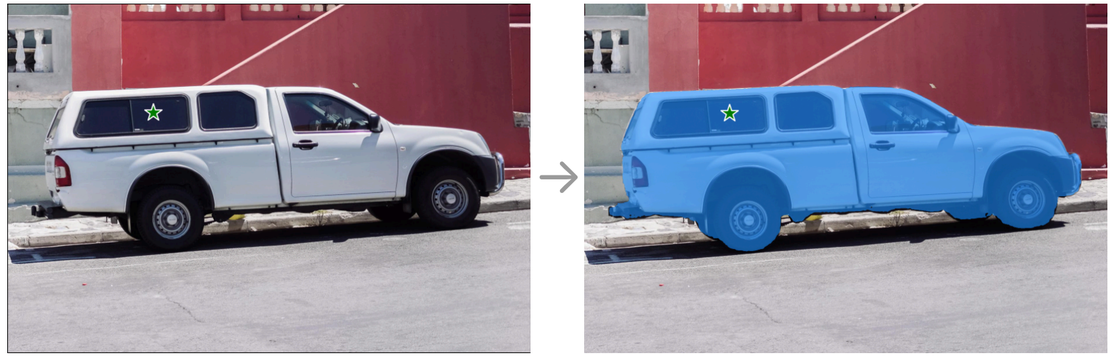
Object detection: YOLO (2016)



Input: an image. Output: a box and a class for every object.

YOLO outputs, Redmon et al. 2016, Fig. 6

Promptable segmentation: SAM (2023)



Input: an image and a click. Output: the object's mask.

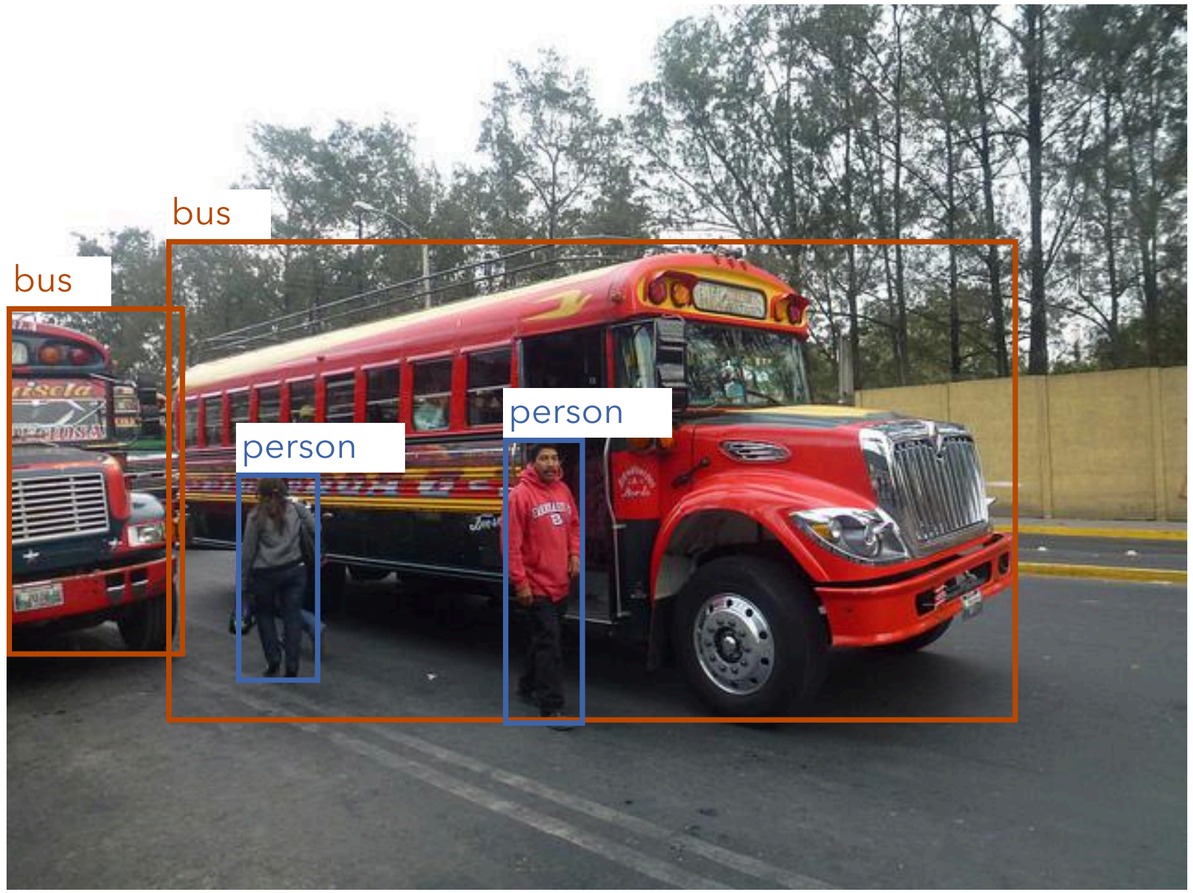
SAM outputs, official SAM notebook

For each application, four parts:

1 Data	2 Architecture	3 Loss	4 Evaluation
what is labelled?	which layers?	what is penalised?	how is success measured?

Redmon et al., You Only Look Once, CVPR 2016. Kirillov et al., Segment Anything, ICCV 2023.

Object detection: what and where?



Selected human annotations from COCO.

Which objects are present?

Where is each object?

Training labels

A class and a bounding box
for each labelled object.

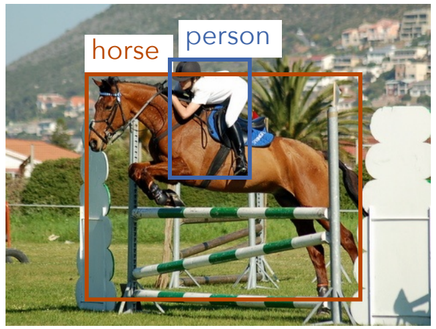
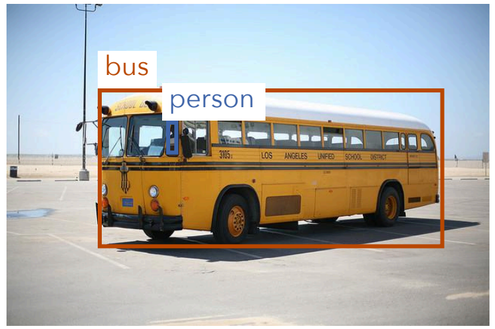
Detector output

Boxes, classes and scores.

Lin et al., Microsoft COCO, ECCV 2014. Image 532 via the official Ultralytics COCO128 release.

Detection datasets

People annotate object classes and locations in photographs.

	PASCAL VOC 2007	COCO 2017
One image and its labels		
Object classes	20	80
Training / validation	5,011 combined	118,287 train; 5,000 val
Detection labels	Classes and boxes	Classes and boxes

YOLO's VOC 2007 experiment trains on VOC 2007 + 2012 train/val: 16,551 images.

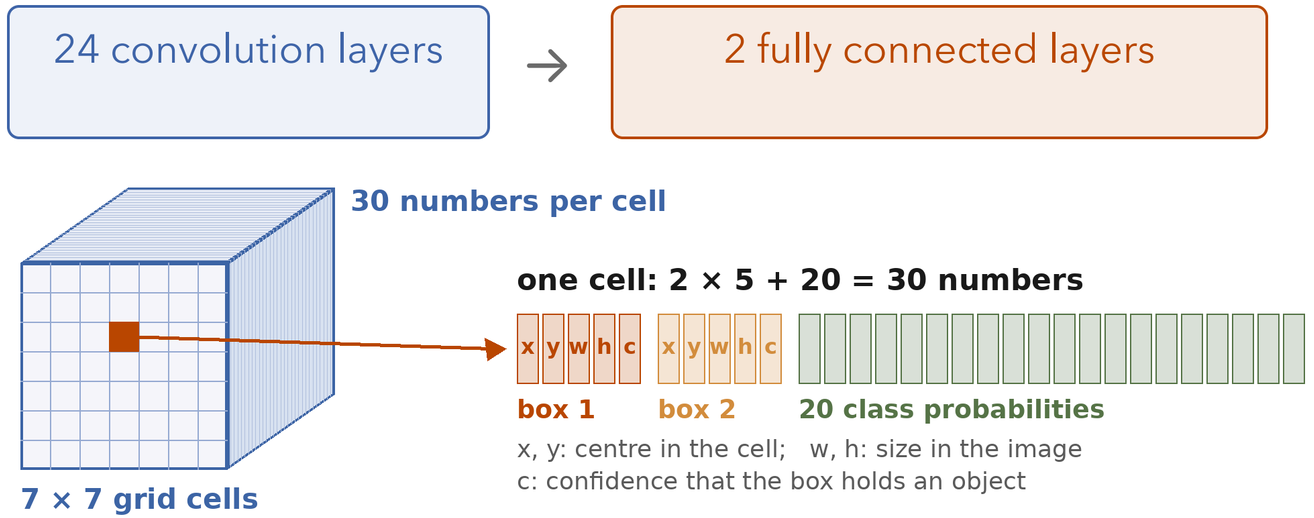
PASCAL VOC 2007 development kit and challenge website. COCO 2017 dataset and annotation documentation.

YOLO: one stage, a grid of predictions



VOC image 000017, resized to a square.

One network predicts all grid cells together.



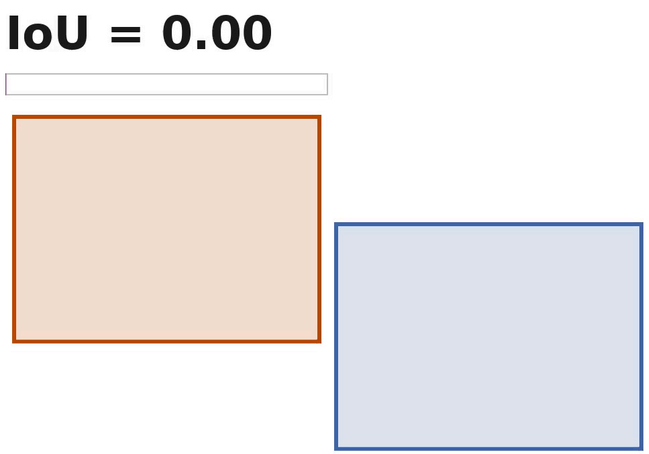
$$7 \times 7 \times (2 \times 5 + 20) = 1{,}470$$

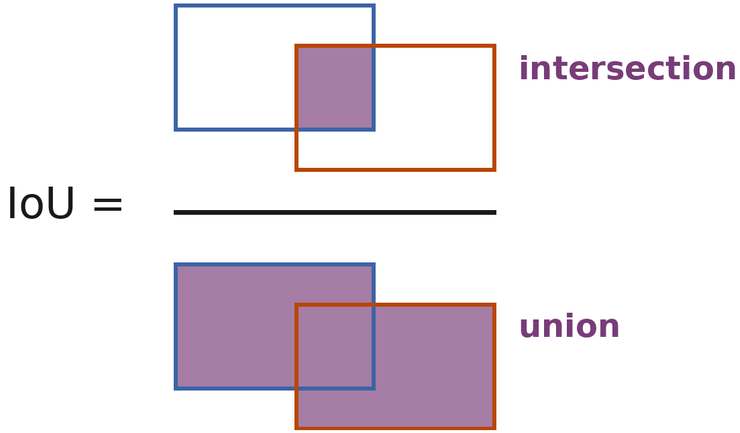
7 × 7 cells × 2 boxes = 98 candidate boxes per image.

Redmon et al., You Only Look Once, CVPR 2016, Fig. 2–3. PASCAL VOC 2007 image 000017.

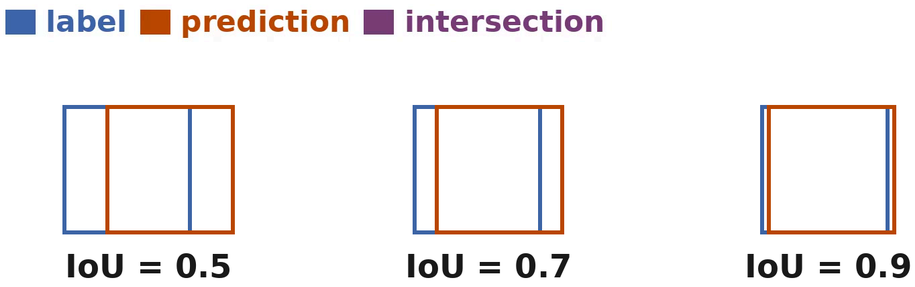
Intersection over union (IoU)

IoU scores how well a predicted box overlaps a labelled box.





1: identical boxes. 0: no overlap.



IoU 0.5 still allows a large shift.

YOLO uses IoU three times

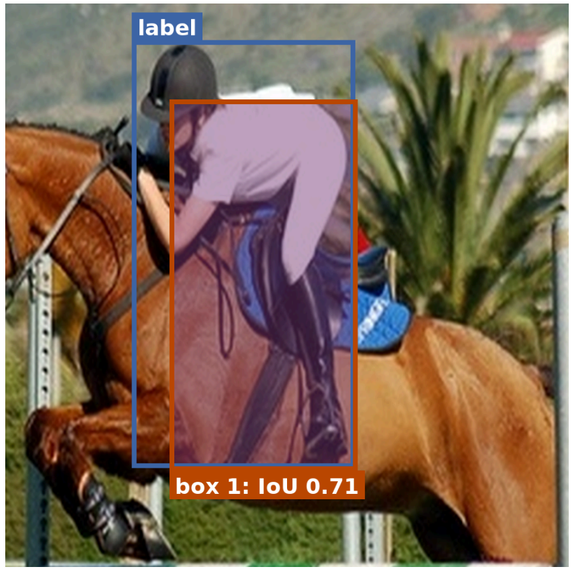
1. **Training:** pick the box predictor that learns the object
2. **NMS:** remove overlapping duplicates
3. **Evaluation:** decide whether a detection is correct

Teaching sequence after MIT 6.8300 (2024), L13. Redmon et al., You Only Look Once, CVPR 2016. PASCAL VOC devkit.

YOLO: one training iteration

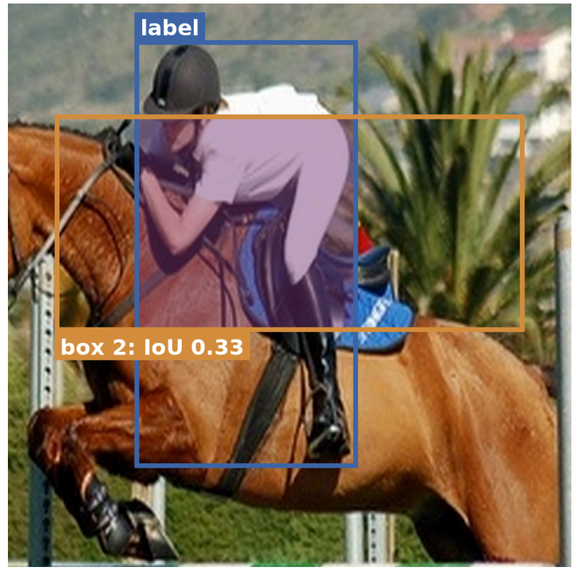


2a. The label's centre (dot) lies in this cell



2b. That cell's **box 1**: IoU 0.71

vs



2b. That cell's **box 2**: IoU 0.33

One training iteration, for one image:

1. Forward pass: 49 cells × 2 boxes = 98 predicted boxes, plus one class vector per cell.
2. For each labelled object:
   - **a.** find the cell that contains its centre;
   - **b.** compute the IoU of that cell's two predicted boxes with the label;
   - **c.** the box with the higher IoU (**box 1**) learns the object: box → label, confidence → IoU; the cell's class → person.
3. Every other box (**box 2**, and both boxes in all cells without an object centre) learns confidence 0.
4. Compute the loss (next slide), backpropagate, update. Next iteration the boxes have moved, so 2b is redone.

Redmon et al., You Only Look Once, CVPR 2016, §2.2. PASCAL VOC 2007 image 000017; predicted boxes illustrative.

YOLO: loss function

Training turns the labels into a target for every number the network outputs.

	box 1 (responsible)					box 2					20 class probabilities																								
	x	y	w	h	c	x	y	w	h	c																										
Person's cell row 2, column 3	0.38	0.50	0.20	0.38	IoU	–	–	–	–	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0							
																								person											
Cell with no object 47 of the 49 cells	–	–	–	–	0	–	–	–	–	0	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–

Grey entries are not in the loss.

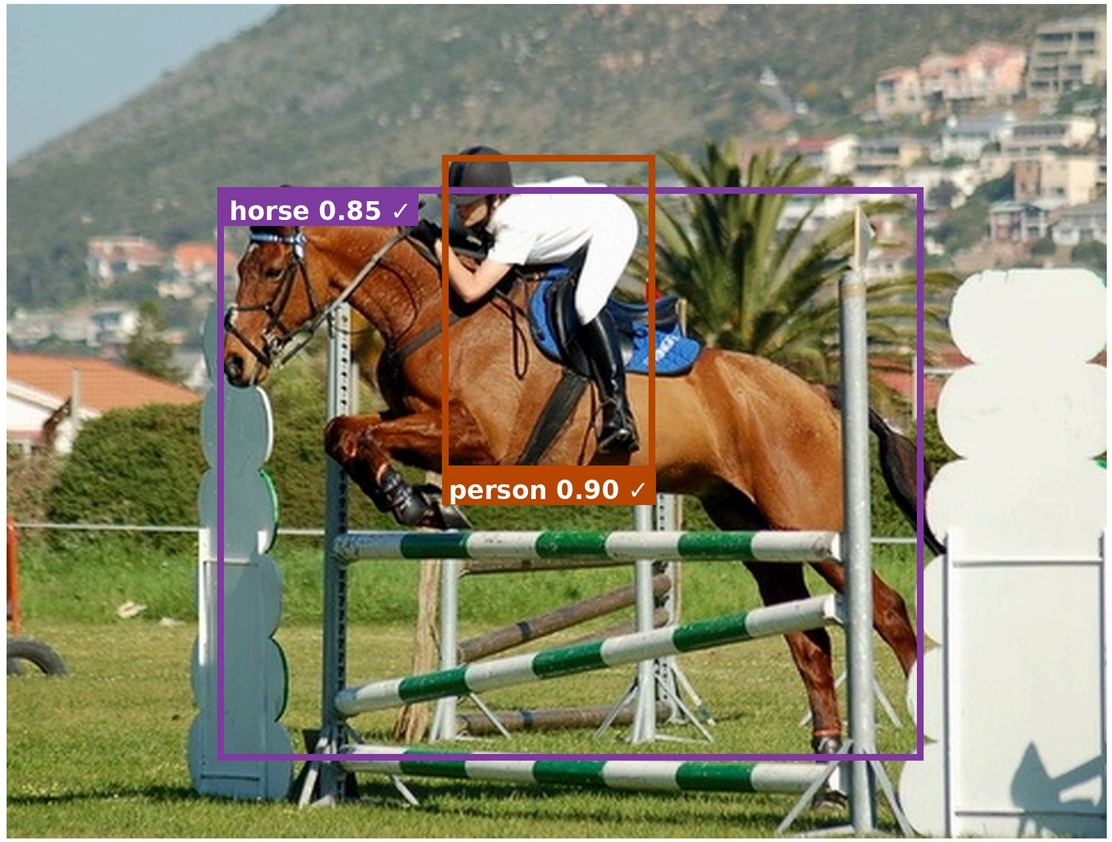
Loss = sum of weight × (output − target)² over the entries with a target:

weight	entries	target
× 5	x, y, √w, √h of each responsible box	the labelled box
× 1	c of each responsible box	its IoU with the label
× 0.5	c of every other box	0
× 1	class probabilities, cells with an object	1 for the true class, else 0

Redmon et al., You Only Look Once, CVPR 2016, §2.2. Full historical objective in the appendix.

Non-maximum suppression (NMS)

Neighbouring cells, and both boxes of a cell, can fire on the same object. NMS keeps one box per object.



At test time, for all 98 boxes:

1. Compute scores.

2. Drop scores below a small threshold (person 0.08).

3. For each class separately, repeat:
   a. keep the highest-scoring remaining box;
   b. delete same-class boxes with IoU > 0.5 to it.

Person: keep 0.90; delete 0.75 (IoU 0.72) and 0.45 (IoU 0.66).

Horse: keep 0.85; delete 0.55 (IoU 0.68).

Result: one box per object.

Redmon et al., You Only Look Once, CVPR 2016, §2.4 (NMS adds 2–3% mAP). PASCAL VOC 2007 image 000017; boxes and scores illustrative.

Detection: evaluation

A detection is correct if its class matches a label and IoU > 0.5.

Blue: label. **Orange:** detection, with class and score.



Correct: TP

Right class, IoU 0.76



Missed: FN

No detection on the horse



Poor overlap: FP

IoU 0.13



Wrong class: FP

Good box, but ‘cow’

Evaluation counts these outcomes as precision and recall:

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Fraction of detections that are correct

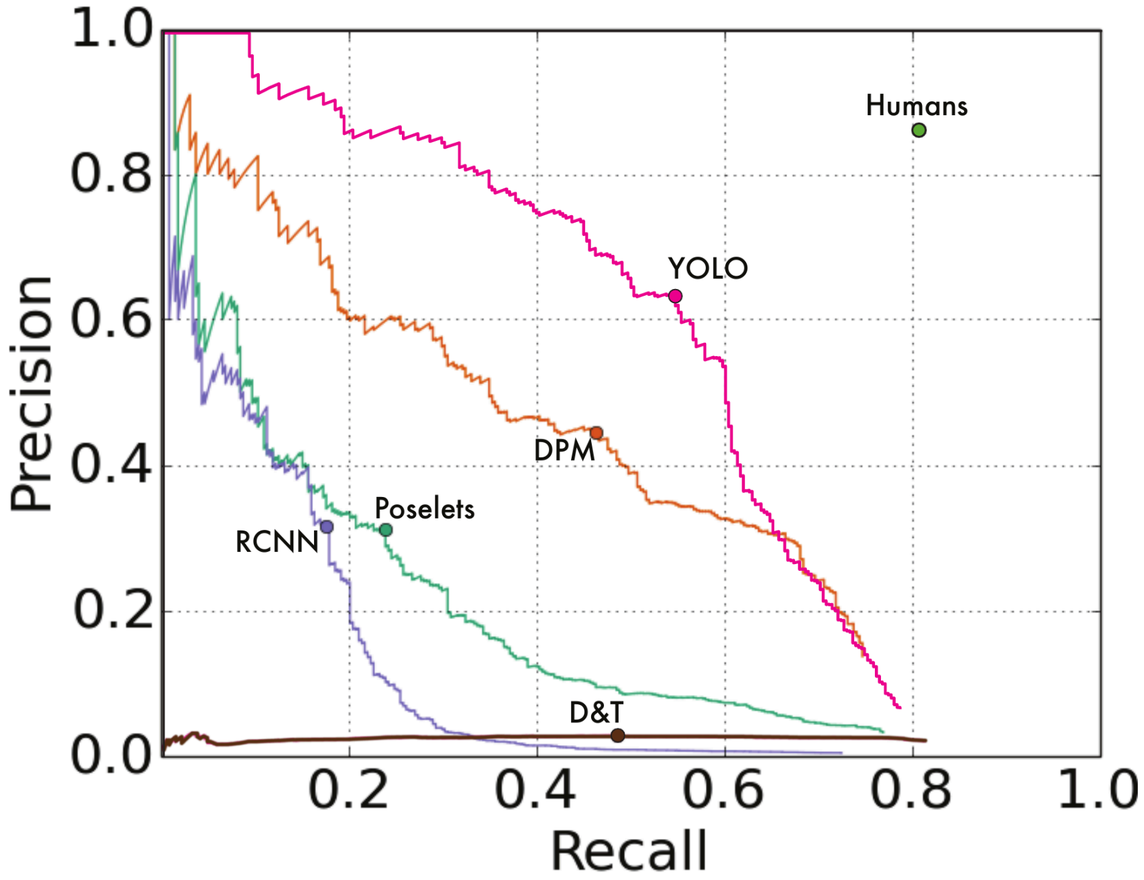
$$\text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Fraction of labelled objects that are found

Teaching sequence after MIT 6.8300 (2024), L13, p.31. PASCAL VOC 2007 image 000017; the detections are constructed for illustration.

Detection: mean average precision (mAP)

IoU > 0.5 decides TP or FP. A score threshold decides which detections are reported.



Redmon et al. 2016, Fig. 5a

AP: the area under one class's precision–recall curve.

Here, person on Picasso: YOLO 53.3, DPM 37.8.

mAP: the mean of AP over all classes.

VOC 2007, 20 classes: YOLO 63.4 mAP at 45 frames/s.

Curves: Redmon et al., You Only Look Once, CVPR 2016, Fig. 5a and Table 1. AP as in the PASCAL VOC devkit; COCO detection evaluation.

Segment Anything (SAM)

From boxes for 20 classes to a mask for any object

Kirillov et al., 2023

	YOLO	SAM
Which objects?	20 VOC classes	whatever the prompt points at
Output	boxes and class scores	masks and quality scores
Labels	16,551 images	11M images, 1.1B masks



One SA-1B image with its masks

Kirillov et al., Segment Anything, ICCV 2023, Fig. 2. Redmon et al., You Only Look Once, CVPR 2016.

SAM: segment what the prompt points at

Input: an image and one or more clicks. Output: masks for the object they point at.



Paper, Fig. 3: each column is one click (green); each row a valid mask.

1. One click can mean several objects: the bird's head, its body, or the whole bird.

2. A single output would average them into a blurred mask that fits none.

3. So SAM returns 3 masks (whole, part, subpart), each with a score: its estimated IoU.

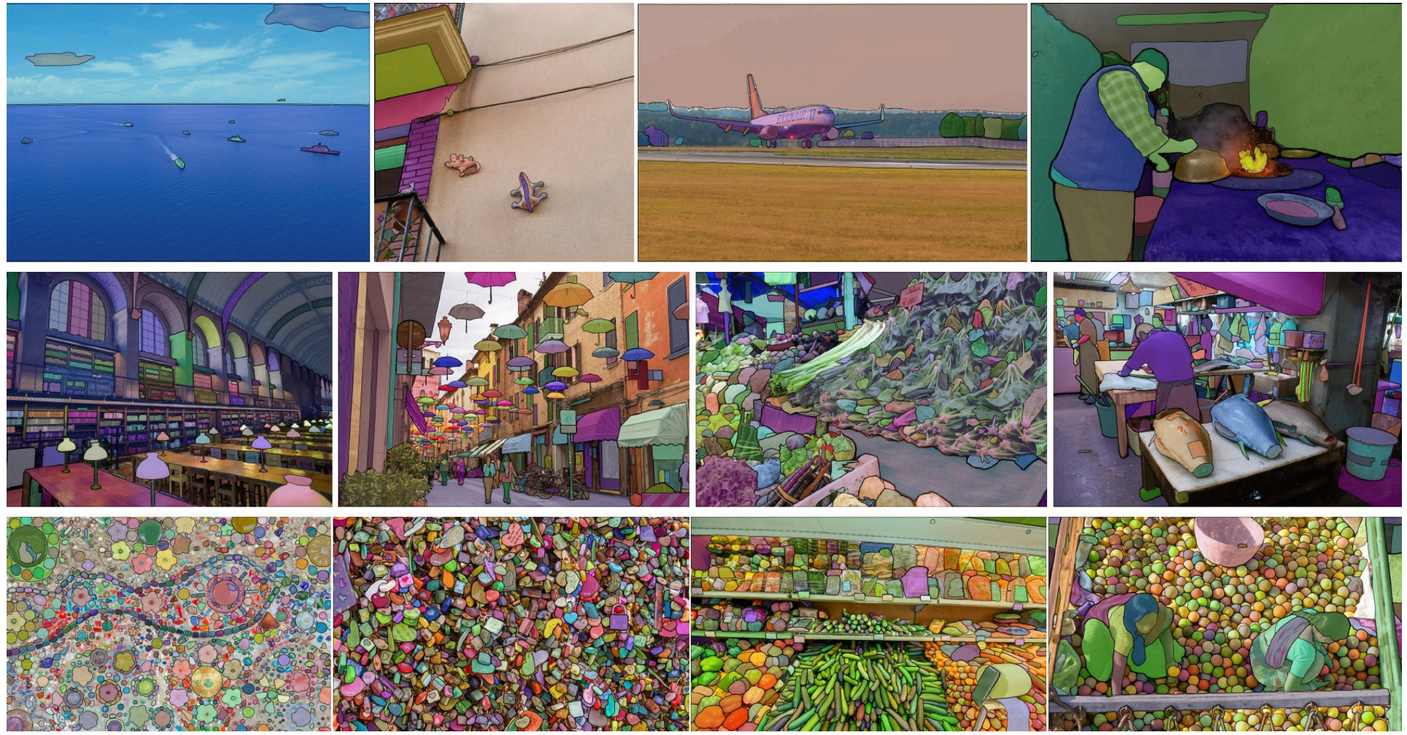
We cover SAM in the same four parts:

1 Data	2 Architecture	3 Loss	4 Evaluation
SA-1B	encoders + decoder	best of 3 masks	23 new datasets

Kirillov et al., Segment Anything, ICCV 2023, §2–3 and Fig. 3.

SAM data: SA-1B

11 million licensed photos with 1.1 billion masks, and no class names.



Paper, Fig. 2: rows with fewer than 50, 200–300, and more than 500 masks per image.

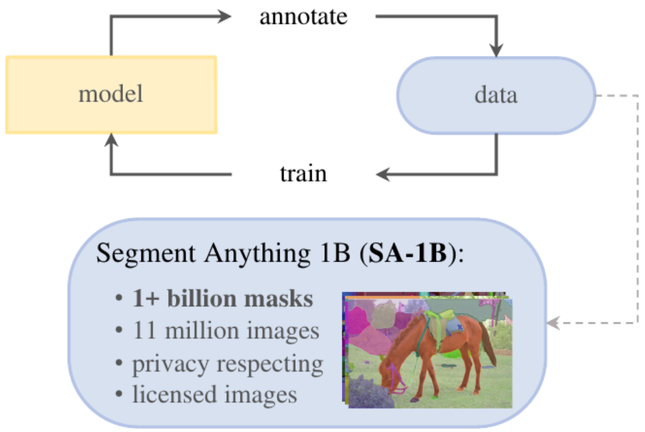
How do you make a billion masks? With the model in the loop: next slide.

1. 11M high-resolution photos
2. 1.1B masks: about 100 per image
3. 99.1% of the masks were made automatically, by SAM
4. 400× more masks than the largest earlier dataset (Open Images)
5. Masks only: no class labels

Kirillov et al., Segment Anything, ICCV 2023, §5 and Fig. 2.

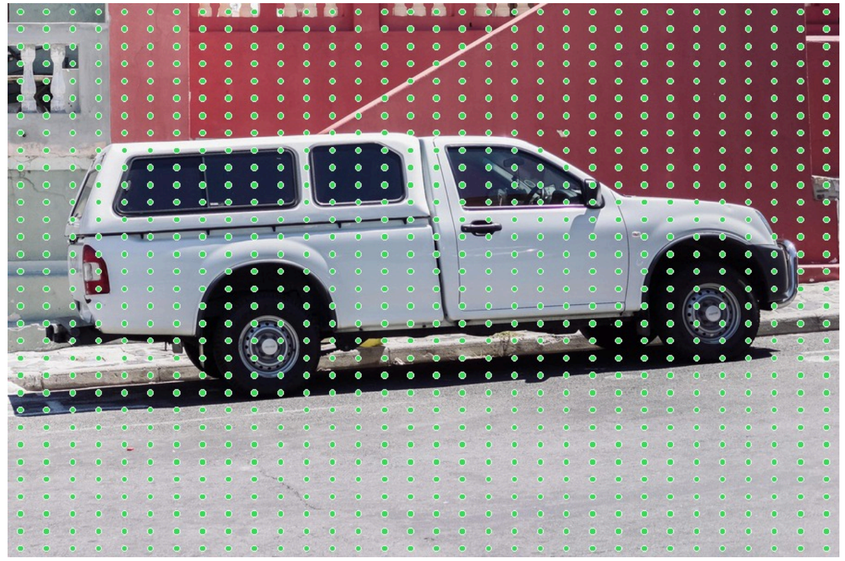
SAM data: the data engine

Annotate with SAM, retrain SAM on the new masks, repeat: the model and the data improve together.



(c) **Data**: data engine (top) & dataset (bottom)

Paper, Fig. 1(c)



Stage 3: a 32 × 32 grid of point prompts on one image

1. Assisted manual

Annotators click; SAM proposes a mask; they fix it with a brush and an eraser.

120k images, 4.3M masks; 34 s → 14 s per mask

2. Semi-automatic

SAM fills in the masks it is confident about; annotators add the objects it missed.

180k images, 5.9M more masks

3. Fully automatic

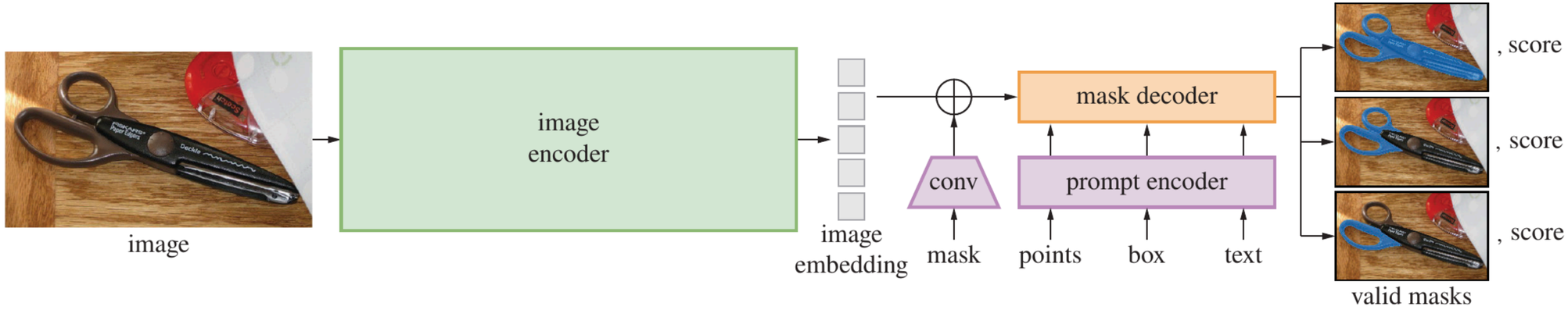
Prompt SAM with a grid of points; keep confident, stable masks; NMS removes duplicates.

11M images, 1.1B masks

Kirillov et al., Segment Anything, ICCV 2023, §4. Truck: official SAM notebook image.

SAM architecture: three parts

A heavy image encoder runs once per image; a light prompt encoder and mask decoder run for every prompt.



Paper, Fig. 4

1. Image encoder: a ViT and 2 convolutions; the image → 64 × 64 = 4,096 image tokens of 256 numbers. Once per image.

2. Prompt encoder: each click → a prompt token of 256 numbers.

3. Mask decoder: 2 transformer layers and a CNN; prompt tokens + image tokens → 3 masks, each with a score. New here: cross-attention.

Kirillov et al., Segment Anything, ICCV 2023, §3 and Fig. 4.

SAM architecture: the prompt encoder

1. image encoder → **2. prompt encoder** → 3. mask decoder



the click (star) on the truck

A click becomes a vector of 256 numbers: a prompt token.

click + = the click token

$\mathrm{PE}(x, y)$: where

e_{fg}: foreground

the user picks one:

e_{fg}: foreground, "include this"

e_{bg}: background, "exclude this"

+ learned output tokens, the same for every image:

mask 1 mask 2 mask 3 score

1. A click becomes one token: $\mathrm{PE}(x, y)$, a code of where it is, plus a learned vector e_{fg}.

2. e_{fg} and e_{bg} are two learned vectors, trained with SAM. The user picks: foreground = "include this", background = "exclude this".

3. Learned output tokens join: 3 mask tokens and a score token, like ViT's class token. They will become the masks and scores.

Into the decoder: 5 prompt tokens (click, 3 mask, score) and 4,096 image tokens, 256 numbers each.

Kirillov et al., Segment Anything, ICCV 2023, §3 and Appendix A. Vectors drawn as 16 cells for 256 numbers. Truck: official SAM notebook.

SAM's mask decoder: cross-attention

1. image encoder → 2. prompt encoder → **3. mask decoder**

The mask decoder is 2 transformer layers and a CNN. The new ingredient is cross-attention.

Self-attention, as in the transformer part

```
qkv = self.qkv(x)           # nn.Linear(D, 3D)
q, k, v = qkv.chunk(3, dim=-1)
s = q @ k.transpose(-2, -1) * D ** -0.5
a = s.softmax(dim=-1)
y = a @ v
x = x + y                   # every token of x changes
```

Cross-attention

```
q = self.q(x)               # nn.Linear(D, D)
k, v = self.kv(c).chunk(2, dim=-1)  # nn.Linear(D, 2D)
s = q @ k.transpose(-2, -1) * D ** -0.5
a = s.softmax(dim=-1)
y = a @ v
x = x + y                   # only x changes; c is only read
```

Only the first two lines change: q comes from x, k and v from another set c. The scores s are (tokens of x) × (tokens of c), and y has one row per token of x, so only x changes.

SAM runs it both ways in each decoder layer:

	x: asks, so it changes	c: only read (keys and values)
step 2	5 prompt tokens	 4,096 image tokens
step 4	 4,096 image tokens	5 prompt tokens

Each of the 2 decoder layers: self-attention on the prompt tokens, step 2, an MLP, step 4. Then a CNN upsamples the image tokens, and each mask token gives its mask by a dot product with them.

Why not self-attention over all 4,101 tokens? Its 4,096 × 4,096 image-to-image part is the costly bit, and the image encoder has already done it.

Kirillov et al., Segment Anything, ICCV 2023, §3, Appendix A and Fig. 14. Code: PyTorch-style, as in the transformer part; c is the other set.

SAM loss and evaluation: mask overlap

Both use the overlap of a predicted mask with the labelled mask: IoU, as for YOLO's boxes, but over pixels.

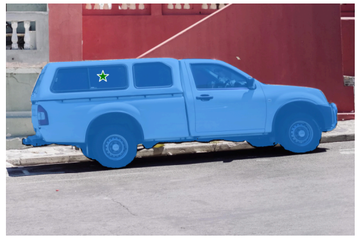
3 Loss



high loss



high loss



lowest loss: trained

1. Per pixel: a cross-entropy (focal loss) plus a soft overlap of mask and label (Dice loss, close to IoU).

2. One click is ambiguous, so only the best of the 3 masks is trained (label here: the whole truck).

3. The score head learns each mask's IoU with the label.

4 Evaluation

4. Metric: IoU of SAM's mask with the labelled mask, averaged over objects (mIoU), plus human ratings

Kirillov et al., Segment Anything, ICCV 2023, §3, §7.1 and Appendix A. Masks: official SAM notebook output.

Detection and promptable segmentation

	Original YOLO	SAM
Task	Find objects from a learned set of classes.	Segment the object selected by a prompt.
Output	Classes, boxes and scores	Pixel masks and mask quality
1 Data	VOC: human class and box annotations	SA-1B: masks generated through a data engine
2 Architecture	CNN with a fixed grid of box and class predictions.	Cache ViT image features. Decode each prompt.
3 Loss	Location, confidence and class targets	Focal + Dice, with a separate mask-quality loss
4 Evaluation	mAP: 63.4 on VOC 2007, at 45 frames per second	mIoU from one click on 23 datasets; human ratings

Redmon et al., You Only Look Once, CVPR 2016. Kirillov et al., Segment Anything, ICCV 2023.